

Linear Regression: Pipelines, Regularization, and Online Learning

Course: Machine Learning Labs

Lab: 01 — Linear Regression

Table of Contents

1. Setup and Imports
 2. Introduction
 3. Part 1 — Sklearn Pipelines with Ridge Regression
 - 3.1 Dataset: California Housing
 - 3.2 Why Pipelines?
 - 3.3 Building the Ridge Pipeline
 - 3.4 Hyperparameter Tuning with GridSearchCV
 - 3.5 Model Evaluation
 - Exercise 1.1 — Effect of Polynomial Degree
 - Exercise 1.2 — Ridge vs. Lasso
 4. Part 2 — Online Learning with SGDRegressor
 - 4.1 When Data Doesn't Fit in Memory
 - 4.2 Dataset: Year Prediction MSD
 - 4.3 SGDRegressor and `partial_fit`
 - Exercise 2.1 — Implement the Mini-Batch Training Loop
 - Exercise 2.2 — Plot the Convergence Curve
 - Exercise 2.3 — Multi-Epoch Training
 - Exercise 2.4 — Learning Rate Exploration
 - Exercise 2.5 — Learning Rate Schedules
 5. Real-World Considerations
 6. Summary and Key Takeaways
-

Estimated Time: 3–4 hours

Prerequisites: Python, NumPy, Pandas, introductory supervised learning



Resolution

After the deadline, you will find the resolution at the following Notion link:

https://app.notion.com/p/R-Reg-lineal-python-intro-2780573743578010af37de77f8e24bee?source=copy_link

Remember that there are several ways to solve the proposed activities in the labs. Use the provided resolution as a reference. If the resolution link is not public, request access at the same Notion link.

Deliverables

Don't forget to duplicate this notebook to be able to edit: File -> Save a copy in Drive

In this lab, you do not need to write a report in a separate document. Please complete all required activities within this Google Colab notebook. Remember that a notebook allows you to enter text elements similarly to a document processor. Once the proposed activities are completed, you must submit the following on the platform:

1. A PDF file generated in Google Colab from the menu "File" -> "Print".
 2. The public Google Colab link. To do so, go to the share button and change the sharing settings to "Anyone with the link".
-

1. Setup and Imports

```
In [1]: # Install / upgrade packages when running in Google Colab
try:
    import google.colab
    IN_COLAB = True
    import subprocess
    subprocess.run(['pip', 'install', '--quiet', '--upgrade',
                    'scikit-learn>=1.3', 'seaborn>=0.12'], check=True)
    print('Colab environment ready.')
except ImportError:
    IN_COLAB = False
    print('Local environment detected.')
```

Colab environment ready.

```
In [2]: import os
import warnings
import urllib.request
import zipfile

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import PolynomialFeatures, StandardScaler, FunctionTransformer
from sklearn.linear_model import Ridge, Lasso, SGDRegressor
from sklearn.model_selection import GridSearchCV, train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error
```

```
import sklearn
print(f'scikit-learn : {sklearn.__version__}')
print(f'NumPy       : {np.__version__}')
print(f'Pandas      : {pd.__version__}')

warnings.filterwarnings('ignore')
```

```
scikit-learn : 1.9.1
NumPy       : 2.1.3
Pandas      : 2.2.3
```

```
In [3]: # — Reproducibility —————
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# — Part 1: Pipeline / GridSearch —————
TEST_SIZE = 0.2
CV_FOLDS = 5
POLY_DEGREE = 3          # Exercise 1.1: try 1 or 3
ALPHAS = np.logspace(-4, 4, 30)

# — Part 2: Online Learning —————
CHUNK_SIZE = 5_000
N_EPOCHS = 10
MSD_TRAIN_ROWS = 463_715 # official UCI split: first 463,715 rows are training
MSD_TEST_ROWS = 51_630  # last 51,630 rows are test (total = 515,345)
MSD_DATA_FILE = 'YearPredictionMSD.txt'
MSD_URL = ('https://archive.ics.uci.edu/ml/machine-learning-databases'
           '/00203/YearPredictionMSD.txt.zip')

# — Plotting defaults —————
os.makedirs('assets', exist_ok=True)
plt.rcParams.update({
    'figure.figsize' : (11, 5),
    'axes.spines.top' : False,
    'axes.spines.right' : False,
    'font.size' : 11,
})
COLORS = ['steelblue', 'tomato', 'seagreen', 'darkorange', 'mediumpurple']

print('Configuration ready.')
```

Configuration ready.

2. Introduction

Linear regression is the foundation of supervised machine learning. Despite its simplicity, it remains one of the most widely deployed models in industry — from predicting house prices to forecasting energy demand. Mastering it means mastering the core ideas that every more complex model builds on: loss functions, gradient descent, regularization, and generalization.

This lab covers two perspectives on linear regression:

Part 1 — Sklearn Pipelines with Ridge Regression (instructor walkthrough)

You will build a complete machine-learning pipeline that chains polynomial feature expansion, feature scaling, and Ridge regression. You will use `GridSearchCV` to tune the regularization strength λ (alpha) and visualize the classic bias–variance tradeoff through a train/validation error curve.

Part 2 — Online Learning with SGDRegressor (student activity)

You will train a linear model on a dataset that is too large to hold in memory at once — the **Year Prediction MSD** dataset, a subset of the Million Song Dataset with 515 K songs and 90 audio features. You will use `SGDRegressor.partial_fit` to process the data in mini-batches, monitor convergence, and experimentally discover how the learning rate drives convergence or catastrophic divergence.

Learning Objectives

By the end of this lab you will be able to:

1. Explain why preprocessing steps must be wrapped in a `Pipeline` to prevent data leakage during cross-validation.
2. Build a multi-step sklearn `Pipeline` combining feature engineering, scaling, and a regularized estimator.
3. Use `GridSearchCV` to perform principled hyperparameter search and interpret the resulting validation curve.
4. Describe the difference between batch, stochastic, and mini-batch gradient descent.
5. Implement online / incremental learning with `SGDRegressor.partial_fit` on chunked data.
6. Diagnose learning-rate divergence and understand the role of learning-rate schedules.

3. Part 1 — Sklearn Pipelines with Ridge Regression

3.1 Dataset: California Housing

We use the **California Housing** dataset, derived from the 1990 US Census. Each row represents one census block group; the task is to predict the **median house value** (in units of \$100,000).

Feature	Description
MedInc	Median household income in block
HouseAge	Median house age in block
AveRooms	Average number of rooms
AveBedrms	Average number of bedrooms

Feature	Description
Population	Block population
AveOccup	Average occupancy per household
Latitude	Block latitude
Longitude	Block longitude

Target: MedHouseVal — median house value in \$100 K (range 0.15 – 5.0).

```
In [4]: housing = fetch_california_housing(as_frame=True)
df = housing.frame
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

print(f'Samples : {df.shape[0]:,}')
print(f'Features : {X.shape[1]}')
print(f'Target : MedHouseVal min={y.min():.2f} max={y.max():.2f} mean={y.mean():.2f}')
print()
display(df.describe().round(2))
```

Samples : 20,640

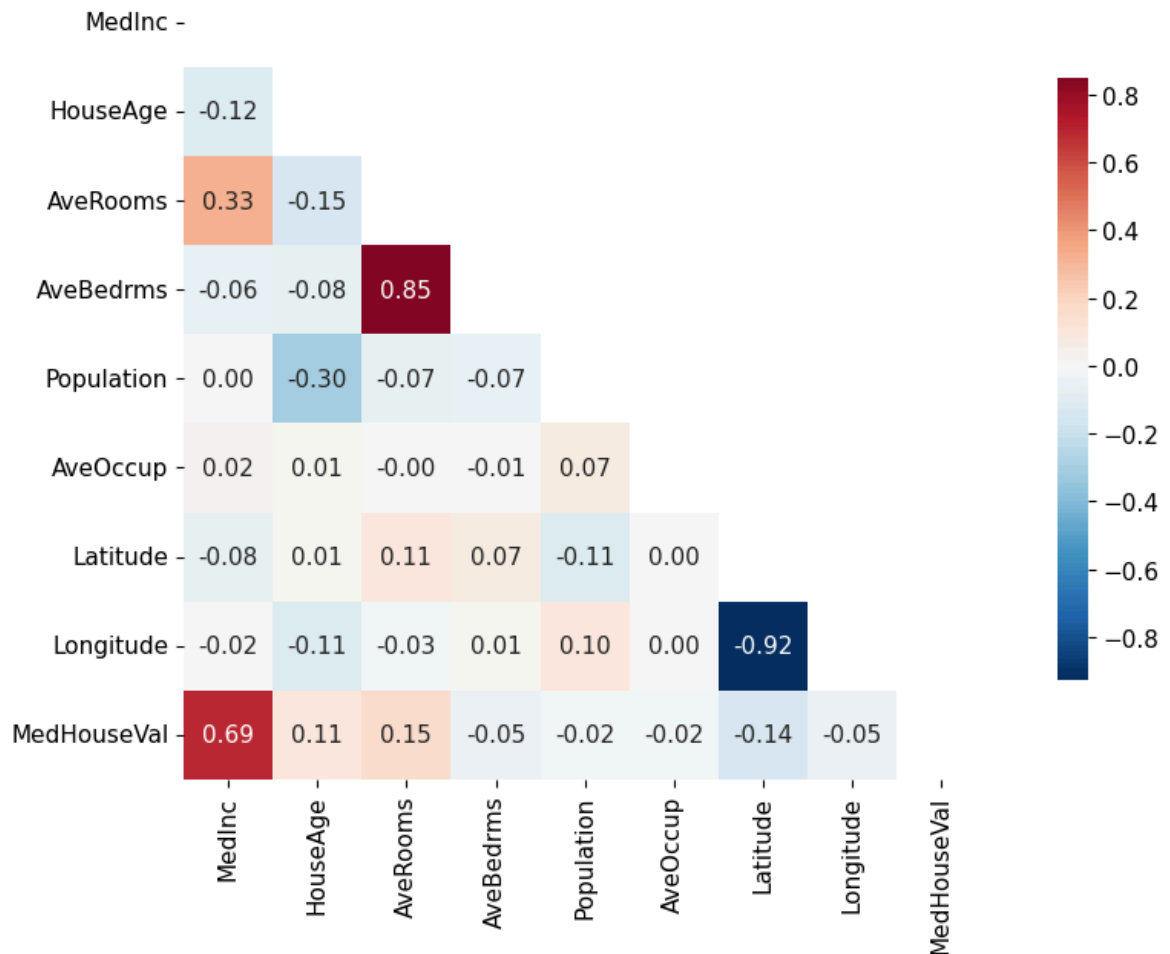
Features : 8

Target : MedHouseVal min=0.15 max=5.00 mean=2.07

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
count	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00
mean	3.87	28.64	5.43	1.10	1425.48	3.07	35.63	-120.22
std	1.90	12.59	2.47	0.47	1132.46	10.39	2.14	1.44
min	0.50	1.00	0.85	0.33	3.00	0.69	32.54	-122.50
25%	2.56	18.00	4.44	1.01	787.00	2.43	33.93	-122.23
50%	3.53	29.00	5.23	1.05	1166.00	2.82	34.26	-122.02
75%	4.74	37.00	6.05	1.10	1725.00	3.28	37.71	-121.89
max	15.00	52.00	141.91	34.07	35682.00	1243.33	41.95	-114.80

```
In [5]: fig, ax = plt.subplots(figsize=(10, 7))
mask = np.triu(np.ones_like(df.corr(), dtype=bool))
sns.heatmap(
    df.corr(), mask=mask, annot=True, fmt='.2f',
    cmap='RdBu_r', center=0, square=True, ax=ax,
    cbar_kws={'shrink': 0.75}
)
ax.set_title('Feature Correlation Matrix', pad=12)
plt.tight_layout()
plt.savefig('assets/california_correlation.png', dpi=120, bbox_inches='tight')
plt.show()
```

Feature Correlation Matrix



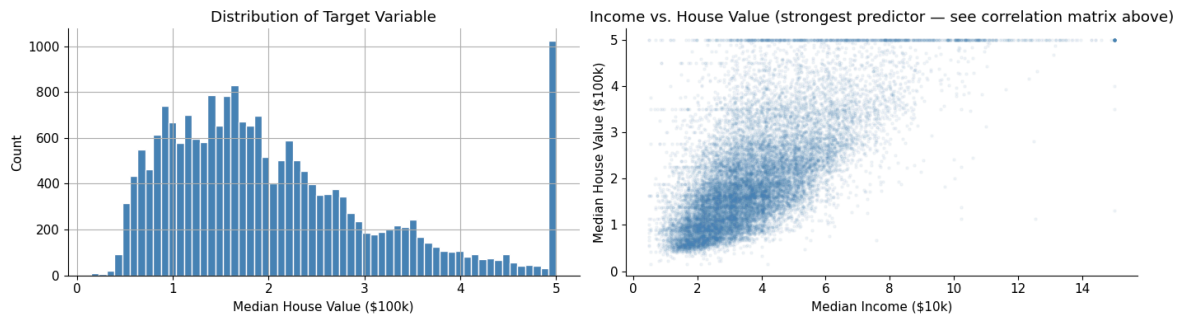
Conclusion from the correlation matrix: MedInc (median income) has the highest correlation with the target (MedHouseVal) at ≈ 0.69 , well above the next strongest predictor AveRooms (≈ 0.15 after accounting for sign). The scatter plot below focuses on this dominant feature and reveals a non-linear curve — motivating polynomial feature expansion in Section 3.3.

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(14, 4))

y.hist(bins=60, ax=axes[0], color=COLORS[0], edgecolor='white')
axes[0].set_xlabel('Median House Value ($100k)')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Target Variable')

axes[1].scatter(df['MedInc'], y, alpha=0.06, s=4, color=COLORS[0])
axes[1].set_xlabel('Median Income ($10k)')
axes[1].set_ylabel('Median House Value ($100k)')
axes[1].set_title('Income vs. House Value (strongest predictor – see correlation)')

plt.tight_layout()
plt.savefig('assets/california_eda.png', dpi=120, bbox_inches='tight')
plt.show()
```



The \$500k Ceiling: Censored Data

The histogram above shows a visible spike at 500k (5.0 in 100k units). This is **not** a market phenomenon — it is a **data collection artifact**. In the 1990 census, median house values above 500k were all recorded as exactly 500k. Statistically, these are **right-censored observations**: we know the true value is $\geq$ \$500k, but not by how much.

Training a regression model on censored targets creates systematic bias: the model is penalized for every block whose true value exceeds \$500k, yet it has no signal to learn the correct magnitude. The result is a distorted loss landscape near the ceiling.

Decision: We filter out the ~965 censored rows ($\approx 4.7\%$ of data) and restrict the model to the 15k–500k range. This is a **data-scope decision**, not data cleaning — the removed rows are not wrong, just incomplete.

```
In [7]: n_total = len(df)
n_capped = (df['MedHouseVal'] >= 5.0).sum()
print(f"Censored rows (MedHouseVal == $500k): {n_capped} / {n_total} ({n_capped}

df = df[df['MedHouseVal'] < 5.0].copy()
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

print(f"Dataset after filtering: {len(df):,} rows")
print(f"Target range now: ${y.min() * 100_000:,.0f} - ${y.max() * 100_000:,.0f}")
```

Censored rows (MedHouseVal == \$500k): 992 / 20640 (4.8%)

Dataset after filtering: 19,648 rows

Target range now: \$14,999 – \$499,100

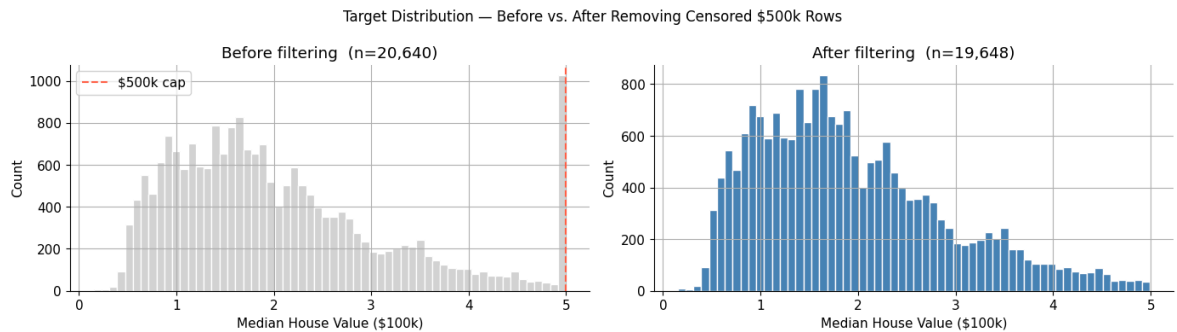
```
In [8]: fig, axes = plt.subplots(1, 2, figsize=(14, 4))

housing.frame['MedHouseVal'].hist(bins=60, ax=axes[0], color='lightgray', edgecolor='black')
axes[0].axvline(5.0, color=COLORS[1], linestyle='--', lw=1.5, label='$500k cap')
axes[0].set_xlabel('Median House Value ($100k)')
axes[0].set_ylabel('Count')
axes[0].set_title('Before filtering (n=20,640)')
axes[0].legend()

y.hist(bins=60, ax=axes[1], color=COLORS[0], edgecolor='white')
axes[1].set_xlabel('Median House Value ($100k)')
axes[1].set_ylabel('Count')
axes[1].set_title(f'After filtering (n={len(df):,})')

plt.suptitle('Target Distribution — Before vs. After Removing Censored $500k Row')
plt.tight_layout()
```

```
plt.savefig('assets/target_distribution_filtered.png', dpi=120, bbox_inches='tight')
plt.show()
```



3.1.1 Feature Skewness and Log Transforms

Raw features are rarely on a nice, symmetric scale — and that matters for linear models:

1. **Outliers dominate OLS.** Squared-error loss amplifies extreme values. AveOccup has a maximum of 1243 vs. a median of ~2.8 — a single such block can pull the fit away from the rest of the data.
2. **Log-linear relationships.** If house value grows *proportionally* with income (rather than additively), the true relationship is linear in $\log(\text{income})$. A model fit on raw income will struggle to capture this curve.

$\log_{1p}(x) = \log(1 + x)$ compresses right tails and safely handles zero values.

Rule of thumb: $|\text{skewness}| > 1 \rightarrow$ consider a log transform.

```
In [9]: LOG_FEATURES = ['MedInc', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup']
PASS_FEATURES = [c for c in X.columns if c not in LOG_FEATURES]

# Compute skewness for all features
feature_skewness = X.skew().sort_values(ascending=False)
print('Feature skewness (sorted):')
print(feature_skewness.round(2).to_string())
print()
print('Features to log-transform (|skew| > 1):', LOG_FEATURES)
print('Features to passthrough          : ', PASS_FEATURES)

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
axes = axes.flatten()

for i, col in enumerate(X.columns):
    skew_val = X[col].skew()
    color = COLORS[1] if abs(skew_val) > 1 else COLORS[0]
    axes[i].hist(X[col], bins=60, color=color, edgecolor='white', alpha=0.8)
    axes[i].set_title(f'{col}\nskew = {skew_val:.2f}', fontsize=9)
    axes[i].set_xlabel(col, fontsize=8)
    axes[i].set_ylabel('Count', fontsize=8)

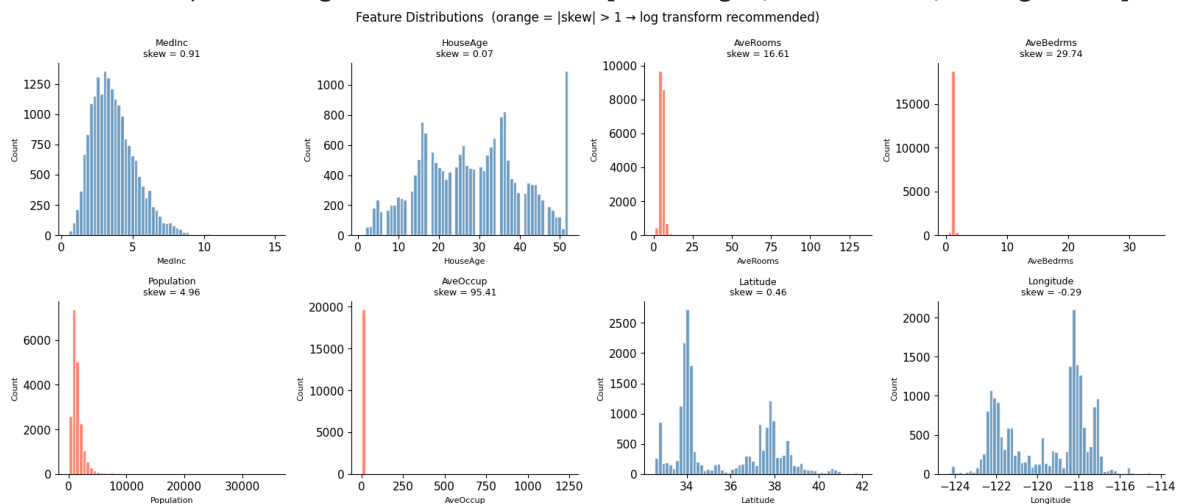
plt.suptitle('Feature Distributions (orange = |skew| > 1 → log transform recomm
plt.tight_layout()
plt.savefig('assets/feature_distributions.png', dpi=120, bbox_inches='tight')
plt.show()
```

Feature skewness (sorted):

AveOccup	95.41
AveBedrms	29.74
AveRooms	16.61
Population	4.96
MedInc	0.91
Latitude	0.46
HouseAge	0.07
Longitude	-0.29

Features to log-transform ($|\text{skew}| > 1$): ['MedInc', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup']

Features to passthrough : ['HouseAge', 'Latitude', 'Longitude']

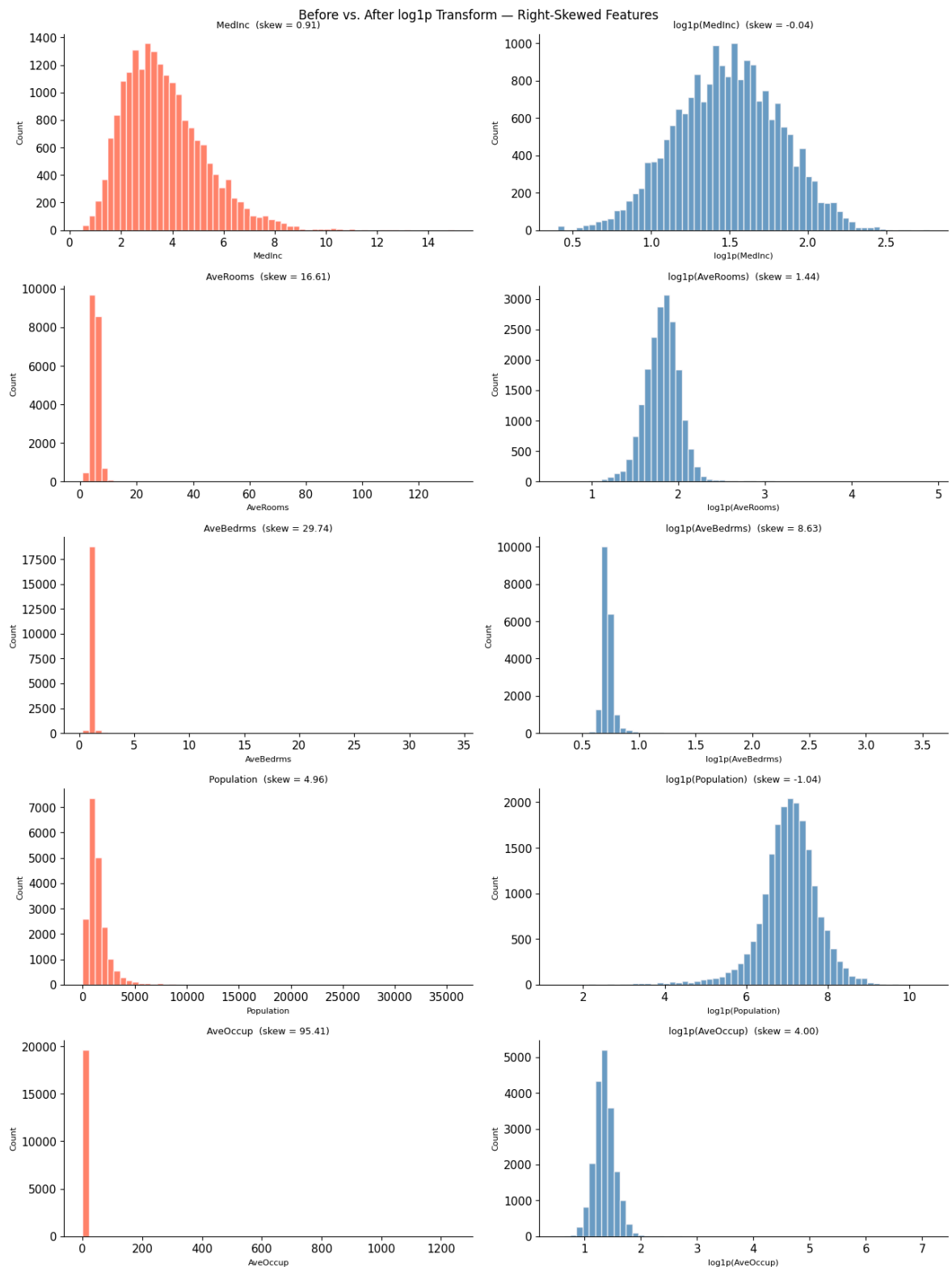


```
In [10]: fig, axes = plt.subplots(len(LOG_FEATURES), 2, figsize=(13, 3.5 * len(LOG_FEATURES)))

for i, col in enumerate(LOG_FEATURES):
    axes[i, 0].hist(X[col], bins=60, color=COLORS[1], edgecolor='white', alpha=0.5)
    axes[i, 0].set_title(f'{col} (skew = {X[col].skew():.2f})', fontsize=9)
    axes[i, 0].set_xlabel(col, fontsize=8)
    axes[i, 0].set_ylabel('Count', fontsize=8)

    log_col = np.log1p(X[col])
    axes[i, 1].hist(log_col, bins=60, color=COLORS[0], edgecolor='white', alpha=0.5)
    axes[i, 1].set_title(f'log1p({col}) (skew = {log_col.skew():.2f})', fontsize=9)
    axes[i, 1].set_xlabel(f'log1p({col})', fontsize=8)
    axes[i, 1].set_ylabel('Count', fontsize=8)

plt.suptitle('Before vs. After log1p Transform – Right-Skewed Features', fontsize=12)
plt.tight_layout()
plt.savefig('assets/log_transforms.png', dpi=120, bbox_inches='tight')
plt.show()
```



Remarks

- All five features become near-symmetric after $\log_{1p}$.
- The ColumnTransformer in Section 3.3 applies this automatically inside the pipeline.

3.2 Why Pipelines? Preventing Data Leakage

A common mistake is to fit the scaler (or any preprocessing step) on the **entire** dataset before splitting into train and test — or before cross-validation folds are formed. This is

data leakage: the scaler learns the mean and standard deviation of the held-out samples, giving an overly optimistic evaluation.

```
# ❌ WRONG – scaler sees test data
scaler.fit(X_all)          # BUG: uses test statistics
X_scaled = scaler.transform(X_all)
X_train, X_test = train_test_split(X_scaled, ...)

# ✅ RIGHT – Pipeline re-fits the scaler inside each CV fold
pipe = Pipeline([('scaler', StandardScaler()), ('ridge',
Ridge())])
cross_val_score(pipe, X_train, y_train, cv=5)
```

With a `Pipeline`, every call to `cross_val_score` or `GridSearchCV` will:

1. Split data into train/validation folds.
2. Call `fit_transform` on the training fold (scaler learns from train only).
3. Call `transform` on the validation fold (applies the train statistics).

This guarantees a valid estimate of generalization performance.

```
In [11]: # Train / test split – done once, before any fitting
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE
)
print(f'Train: {X_train.shape[0]:,} samples')
print(f'Test : {X_test.shape[0]:,} samples')

# Quick baseline with a minimal pipeline (no poly features)
baseline_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge()),
])
baseline_cv = -cross_val_score(
    baseline_pipe, X_train, y_train,
    cv=CV_FOLDS, scoring='neg_mean_squared_error', n_jobs=-1
)
baseline_rmse = np.sqrt(baseline_cv.mean())
print(f'\nBaseline CV RMSE (Ridge, no poly): {baseline_rmse:.4f} ($100k)')
```

Train: 15,718 samples

Test : 3,930 samples

Baseline CV RMSE (Ridge, no poly): 0.6353 (\$100k)

3.3 Building the Ridge Pipeline

The income–value scatter (Section 3.1) shows a non-linear curve. We address this at two levels:

Step 1 — Log transforms (`ColumnTransformer`).

As shown in Section 3.1.1, five features (`MedInc`, `AveRooms`, `AveBedrms`,

`Population`, `AveOccup`) are strongly right-skewed. We apply $\log_{10}(x) = \log(1 +$

x) to each. `HouseAge`, `Latitude`, and `Longitude` pass through unchanged — they are either roughly symmetric or represent coordinates without a meaningful log scale.

Step 2 — Polynomial interaction features.

We use `interaction_only=True`, which adds only **pairwise products** (e.g. `log(MedInc)×Latitude`, capturing income–location pricing effects) and drops squared terms. This gives 8 original + $C(8,2) = 28$ interaction terms = **36 features**.

Step 3 — StandardScaler. Zero-mean, unit-variance scaling is required before Ridge, which penalises coefficient magnitude and is sensitive to feature scale.

Step 4 — Ridge regression. L2 penalty $\alpha \cdot \|w\|^2$ stabilises the fit against the many correlated interaction features.

Our full pipeline:

```
raw features → ColumnTransformer → PolynomialFeatures →
StandardScaler → Ridge
(8 features)    log1p(5 features)    (interaction_only:
(zero mean,      (L2-regularized
unit variance)   passthrough(3 feat.)    8 + 28 = 36 feat.)
                    linear model)
```

Why Ridge? Polynomial expansion creates many correlated features. OLS is then numerically unstable and overfits. Ridge's L2 penalty shrinks weights toward zero, reducing variance at the cost of a small bias.

```
In [12]: log_preprocessor = ColumnTransformer([
    ('log', FunctionTransformer(np.log1p, feature_names_out='one-to-one'), LOG_
    ('pass', 'passthrough', PASS_FEATURES),
], verbose_feature_names_out=False)

pipeline = Pipeline([
    ('log_transform', log_preprocessor),
    ('poly', PolynomialFeatures(degree=POLY_DEGREE, include_bias=False, intera
    ('scaler', StandardScaler()),
    ('ridge', Ridge()),
])

print('Log-transform features :', LOG_FEATURES)
print('Passthrough features  :', PASS_FEATURES)

n_poly_features = (
    PolynomialFeatures(degree=POLY_DEGREE, include_bias=False, interaction_only=
    .fit_transform(X_train)
    .shape[1]
)
print(f'\nDegree-{POLY_DEGREE} polynomial expansion: {X_train.shape[1]} → {n_pol
```

```
Log-transform features : ['MedInc', 'AveRooms', 'AveBedrms', 'Population', 'AveOc
cup']
```

```
Passthrough features  : ['HouseAge', 'Latitude', 'Longitude']
```

Degree-3 polynomial expansion: 8 → 164 features

3.4 Hyperparameter Tuning with GridSearchCV

The regularization strength α (called `ridge__alpha` in the pipeline) controls the bias–variance tradeoff:

- **Small α → low regularization:** the model fits training data tightly, potentially overfitting.
- **Large α → strong regularization:** weights are shrunk toward zero, potentially underfitting.
- **Sweet spot:** the α that minimises validation error.

`GridSearchCV` evaluates every candidate α using k-fold cross-validation and returns the one with the best mean validation score. The double underscore syntax

`'ridge__alpha'` refers to the `alpha` parameter of the `ridge` step inside the pipeline.

```
In [13]: param_grid = {'ridge__alpha': ALPHAS}

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=CV_FOLDS,
    scoring='neg_mean_squared_error',
    return_train_score=True,
    n_jobs=1,
    verbose=6,
)

print(f'Fitting {CV_FOLDS}-fold CV over {len(ALPHAS)} alpha values ...')
grid_search.fit(X_train, y_train)
print('Done.')
print(f'\nBest alpha : {grid_search.best_params_["ridge__alpha"]:.4f}')
print(f'Best CV RMSE: {np.sqrt(-grid_search.best_score_):.4f} (improvement over
      f'{baseline_rmse - np.sqrt(-grid_search.best_score_):.4f}'))
```

Fitting 5-fold CV over 30 alpha values ...
Fitting 5 folds for each of 30 candidates, totalling 150 fits
[CV 1/5] END ridge__alpha=0.0001; score=(train=-0.264, test=-0.361) total time=0.2s
[CV 2/5] END ridge__alpha=0.0001; score=(train=-0.258, test=-0.325) total time=0.1s
[CV 3/5] END ridge__alpha=0.0001; score=(train=-0.264, test=-0.313) total time=0.2s
[CV 4/5] END ridge__alpha=0.0001; score=(train=-0.270, test=-0.285) total time=0.2s
[CV 5/5] END ridge__alpha=0.0001; score=(train=-0.264, test=-0.372) total time=0.2s
[CV 1/5] END ridge__alpha=0.00018873918221350977; score=(train=-0.264, test=-0.345) total time= 0.2s
[CV 2/5] END ridge__alpha=0.00018873918221350977; score=(train=-0.259, test=-0.324) total time= 0.1s
[CV 3/5] END ridge__alpha=0.00018873918221350977; score=(train=-0.265, test=-0.312) total time= 0.1s
[CV 4/5] END ridge__alpha=0.00018873918221350977; score=(train=-0.270, test=-0.281) total time= 0.1s
[CV 5/5] END ridge__alpha=0.00018873918221350977; score=(train=-0.264, test=-0.356) total time= 0.1s
[CV 1/5] END ridge__alpha=0.0003562247890262444; score=(train=-0.264, test=-0.332) total time= 0.1s
[CV 2/5] END ridge__alpha=0.0003562247890262444; score=(train=-0.259, test=-0.322) total time= 0.1s
[CV 3/5] END ridge__alpha=0.0003562247890262444; score=(train=-0.265, test=-0.310) total time= 0.1s
[CV 4/5] END ridge__alpha=0.0003562247890262444; score=(train=-0.270, test=-0.276) total time= 0.1s
[CV 5/5] END ridge__alpha=0.0003562247890262444; score=(train=-0.264, test=-0.339) total time= 0.1s
[CV 1/5] END ridge__alpha=0.0006723357536499335; score=(train=-0.265, test=-0.322) total time= 0.2s
[CV 2/5] END ridge__alpha=0.0006723357536499335; score=(train=-0.259, test=-0.320) total time= 0.2s
[CV 3/5] END ridge__alpha=0.0006723357536499335; score=(train=-0.265, test=-0.306) total time= 0.2s
[CV 4/5] END ridge__alpha=0.0006723357536499335; score=(train=-0.271, test=-0.271) total time= 0.1s
[CV 5/5] END ridge__alpha=0.0006723357536499335; score=(train=-0.265, test=-0.323) total time= 0.1s
[CV 1/5] END ridge__alpha=0.0012689610031679222; score=(train=-0.265, test=-0.315) total time= 0.1s
[CV 2/5] END ridge__alpha=0.0012689610031679222; score=(train=-0.260, test=-0.317) total time= 0.2s
[CV 3/5] END ridge__alpha=0.0012689610031679222; score=(train=-0.266, test=-0.301) total time= 0.1s
[CV 4/5] END ridge__alpha=0.0012689610031679222; score=(train=-0.271, test=-0.267) total time= 0.1s
[CV 5/5] END ridge__alpha=0.0012689610031679222; score=(train=-0.265, test=-0.310) total time= 0.2s
[CV 1/5] END ridge__alpha=0.002395026619987486; score=(train=-0.266, test=-0.311) total time= 0.2s
[CV 2/5] END ridge__alpha=0.002395026619987486; score=(train=-0.260, test=-0.314) total time= 0.3s
[CV 3/5] END ridge__alpha=0.002395026619987486; score=(train=-0.266, test=-0.294) total time= 0.3s
[CV 4/5] END ridge__alpha=0.002395026619987486; score=(train=-0.272, test=-0.265) total time= 0.5s

[CV 5/5] END ridge__alpha=0.002395026619987486;; score=(train=-0.266, test=-0.299) total time= 0.4s
[CV 1/5] END ridge__alpha=0.004520353656360241;; score=(train=-0.267, test=-0.307) total time= 0.3s
[CV 2/5] END ridge__alpha=0.004520353656360241;; score=(train=-0.261, test=-0.312) total time= 0.3s
[CV 3/5] END ridge__alpha=0.004520353656360241;; score=(train=-0.267, test=-0.288) total time= 0.3s
[CV 4/5] END ridge__alpha=0.004520353656360241;; score=(train=-0.273, test=-0.263) total time= 0.2s
[CV 5/5] END ridge__alpha=0.004520353656360241;; score=(train=-0.266, test=-0.292) total time= 0.2s
[CV 1/5] END ridge__alpha=0.008531678524172805;; score=(train=-0.267, test=-0.303) total time= 0.2s
[CV 2/5] END ridge__alpha=0.008531678524172805;; score=(train=-0.262, test=-0.310) total time= 0.2s
[CV 3/5] END ridge__alpha=0.008531678524172805;; score=(train=-0.268, test=-0.283) total time= 0.1s
[CV 4/5] END ridge__alpha=0.008531678524172805;; score=(train=-0.273, test=-0.262) total time= 0.1s
[CV 5/5] END ridge__alpha=0.008531678524172805;; score=(train=-0.267, test=-0.288) total time= 0.2s
[CV 1/5] END ridge__alpha=0.01610262027560939;; score=(train=-0.268, test=-0.299) total time= 0.1s
[CV 2/5] END ridge__alpha=0.01610262027560939;; score=(train=-0.263, test=-0.309) total time= 0.1s
[CV 3/5] END ridge__alpha=0.01610262027560939;; score=(train=-0.269, test=-0.280) total time= 0.2s
[CV 4/5] END ridge__alpha=0.01610262027560939;; score=(train=-0.274, test=-0.261) total time= 0.1s
[CV 5/5] END ridge__alpha=0.01610262027560939;; score=(train=-0.268, test=-0.286) total time= 0.2s
[CV 1/5] END ridge__alpha=0.03039195382313198;; score=(train=-0.269, test=-0.294) total time= 0.2s
[CV 2/5] END ridge__alpha=0.03039195382313198;; score=(train=-0.263, test=-0.309) total time= 0.2s
[CV 3/5] END ridge__alpha=0.03039195382313198;; score=(train=-0.269, test=-0.279) total time= 0.2s
[CV 4/5] END ridge__alpha=0.03039195382313198;; score=(train=-0.275, test=-0.261) total time= 0.2s
[CV 5/5] END ridge__alpha=0.03039195382313198;; score=(train=-0.269, test=-0.284) total time= 0.1s
[CV 1/5] END ridge__alpha=0.05736152510448681;; score=(train=-0.270, test=-0.289) total time= 0.1s
[CV 2/5] END ridge__alpha=0.05736152510448681;; score=(train=-0.264, test=-0.309) total time= 0.2s
[CV 3/5] END ridge__alpha=0.05736152510448681;; score=(train=-0.270, test=-0.279) total time= 0.2s
[CV 4/5] END ridge__alpha=0.05736152510448681;; score=(train=-0.276, test=-0.261) total time= 0.1s
[CV 5/5] END ridge__alpha=0.05736152510448681;; score=(train=-0.269, test=-0.285) total time= 0.2s
[CV 1/5] END ridge__alpha=0.1082636733874054;; score=(train=-0.271, test=-0.285) total time= 0.1s
[CV 2/5] END ridge__alpha=0.1082636733874054;; score=(train=-0.265, test=-0.309) total time= 0.2s
[CV 3/5] END ridge__alpha=0.1082636733874054;; score=(train=-0.271, test=-0.280) total time= 0.1s
[CV 4/5] END ridge__alpha=0.1082636733874054;; score=(train=-0.277, test=-0.263) total time= 0.1s

[CV 5/5] END ridge__alpha=0.1082636733874054;; score=(train=-0.270, test=-0.286)
total time= 0.1s
[CV 1/5] END ridge__alpha=0.20433597178569418;; score=(train=-0.272, test=-0.283)
total time= 0.1s
[CV 2/5] END ridge__alpha=0.20433597178569418;; score=(train=-0.266, test=-0.309)
total time= 0.1s
[CV 3/5] END ridge__alpha=0.20433597178569418;; score=(train=-0.273, test=-0.282)
total time= 0.1s
[CV 4/5] END ridge__alpha=0.20433597178569418;; score=(train=-0.278, test=-0.264)
total time= 0.2s
[CV 5/5] END ridge__alpha=0.20433597178569418;; score=(train=-0.271, test=-0.288)
total time= 0.2s
[CV 1/5] END ridge__alpha=0.38566204211634725;; score=(train=-0.274, test=-0.283)
total time= 0.2s
[CV 2/5] END ridge__alpha=0.38566204211634725;; score=(train=-0.268, test=-0.308)
total time= 0.2s
[CV 3/5] END ridge__alpha=0.38566204211634725;; score=(train=-0.274, test=-0.284)
total time= 0.1s
[CV 4/5] END ridge__alpha=0.38566204211634725;; score=(train=-0.280, test=-0.264)
total time= 0.1s
[CV 5/5] END ridge__alpha=0.38566204211634725;; score=(train=-0.273, test=-0.288)
total time= 0.1s
[CV 1/5] END ridge__alpha=0.7278953843983146;; score=(train=-0.276, test=-0.285)
total time= 0.1s
[CV 2/5] END ridge__alpha=0.7278953843983146;; score=(train=-0.271, test=-0.308)
total time= 0.1s
[CV 3/5] END ridge__alpha=0.7278953843983146;; score=(train=-0.277, test=-0.286)
total time= 0.1s
[CV 4/5] END ridge__alpha=0.7278953843983146;; score=(train=-0.282, test=-0.265)
total time= 0.2s
[CV 5/5] END ridge__alpha=0.7278953843983146;; score=(train=-0.275, test=-0.287)
total time= 0.1s
[CV 1/5] END ridge__alpha=1.3738237958832638;; score=(train=-0.279, test=-0.288)
total time= 0.2s
[CV 2/5] END ridge__alpha=1.3738237958832638;; score=(train=-0.274, test=-0.309)
total time= 0.2s
[CV 3/5] END ridge__alpha=1.3738237958832638;; score=(train=-0.280, test=-0.289)
total time= 0.2s
[CV 4/5] END ridge__alpha=1.3738237958832638;; score=(train=-0.285, test=-0.266)
total time= 0.1s
[CV 5/5] END ridge__alpha=1.3738237958832638;; score=(train=-0.278, test=-0.288)
total time= 0.2s
[CV 1/5] END ridge__alpha=2.592943797404667;; score=(train=-0.282, test=-0.291) t
otal time= 0.3s
[CV 2/5] END ridge__alpha=2.592943797404667;; score=(train=-0.277, test=-0.311) t
otal time= 0.2s
[CV 3/5] END ridge__alpha=2.592943797404667;; score=(train=-0.283, test=-0.292) t
otal time= 0.3s
[CV 4/5] END ridge__alpha=2.592943797404667;; score=(train=-0.288, test=-0.268) t
otal time= 0.4s
[CV 5/5] END ridge__alpha=2.592943797404667;; score=(train=-0.282, test=-0.290) t
otal time= 0.4s
[CV 1/5] END ridge__alpha=4.893900918477489;; score=(train=-0.286, test=-0.295) t
otal time= 0.4s
[CV 2/5] END ridge__alpha=4.893900918477489;; score=(train=-0.281, test=-0.315) t
otal time= 0.3s
[CV 3/5] END ridge__alpha=4.893900918477489;; score=(train=-0.287, test=-0.293) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=4.893900918477489;; score=(train=-0.291, test=-0.272) t
otal time= 0.1s

[CV 5/5] END ridge__alpha=4.893900918477489;; score=(train=-0.286, test=-0.293) t
otal time= 0.2s
[CV 1/5] END ridge__alpha=9.236708571873866;; score=(train=-0.289, test=-0.298) t
otal time= 0.2s
[CV 2/5] END ridge__alpha=9.236708571873866;; score=(train=-0.285, test=-0.319) t
otal time= 0.2s
[CV 3/5] END ridge__alpha=9.236708571873866;; score=(train=-0.291, test=-0.294) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=9.236708571873866;; score=(train=-0.295, test=-0.277) t
otal time= 0.1s
[CV 5/5] END ridge__alpha=9.236708571873866;; score=(train=-0.289, test=-0.295) t
otal time= 0.1s
[CV 1/5] END ridge__alpha=17.433288221999874;; score=(train=-0.293, test=-0.301)
total time= 0.1s
[CV 2/5] END ridge__alpha=17.433288221999874;; score=(train=-0.288, test=-0.323)
total time= 0.1s
[CV 3/5] END ridge__alpha=17.433288221999874;; score=(train=-0.294, test=-0.295)
total time= 0.1s
[CV 4/5] END ridge__alpha=17.433288221999874;; score=(train=-0.298, test=-0.281)
total time= 0.1s
[CV 5/5] END ridge__alpha=17.433288221999874;; score=(train=-0.293, test=-0.297)
total time= 0.1s
[CV 1/5] END ridge__alpha=32.90344562312671;; score=(train=-0.296, test=-0.304) t
otal time= 0.2s
[CV 2/5] END ridge__alpha=32.90344562312671;; score=(train=-0.291, test=-0.326) t
otal time= 0.2s
[CV 3/5] END ridge__alpha=32.90344562312671;; score=(train=-0.298, test=-0.296) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=32.90344562312671;; score=(train=-0.302, test=-0.285) t
otal time= 0.1s
[CV 5/5] END ridge__alpha=32.90344562312671;; score=(train=-0.296, test=-0.300) t
otal time= 0.2s
[CV 1/5] END ridge__alpha=62.10169418915616;; score=(train=-0.299, test=-0.306) t
otal time= 0.1s
[CV 2/5] END ridge__alpha=62.10169418915616;; score=(train=-0.294, test=-0.330) t
otal time= 0.1s
[CV 3/5] END ridge__alpha=62.10169418915616;; score=(train=-0.301, test=-0.298) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=62.10169418915616;; score=(train=-0.305, test=-0.288) t
otal time= 0.2s
[CV 5/5] END ridge__alpha=62.10169418915616;; score=(train=-0.299, test=-0.303) t
otal time= 0.2s
[CV 1/5] END ridge__alpha=117.21022975334793;; score=(train=-0.303, test=-0.309)
total time= 0.2s
[CV 2/5] END ridge__alpha=117.21022975334793;; score=(train=-0.297, test=-0.332)
total time= 0.2s
[CV 3/5] END ridge__alpha=117.21022975334793;; score=(train=-0.304, test=-0.301)
total time= 0.2s
[CV 4/5] END ridge__alpha=117.21022975334793;; score=(train=-0.308, test=-0.291)
total time= 0.1s
[CV 5/5] END ridge__alpha=117.21022975334793;; score=(train=-0.303, test=-0.306)
total time= 0.1s
[CV 1/5] END ridge__alpha=221.22162910704503;; score=(train=-0.307, test=-0.312)
total time= 0.2s
[CV 2/5] END ridge__alpha=221.22162910704503;; score=(train=-0.301, test=-0.335)
total time= 0.1s
[CV 3/5] END ridge__alpha=221.22162910704503;; score=(train=-0.308, test=-0.305)
total time= 0.2s
[CV 4/5] END ridge__alpha=221.22162910704503;; score=(train=-0.312, test=-0.293)
total time= 0.1s

```
[CV 5/5] END ridge__alpha=221.22162910704503;; score=(train=-0.307, test=-0.309)
total time= 0.2s
[CV 1/5] END ridge__alpha=417.53189365604004;; score=(train=-0.312, test=-0.316)
total time= 0.1s
[CV 2/5] END ridge__alpha=417.53189365604004;; score=(train=-0.306, test=-0.338)
total time= 0.1s
[CV 3/5] END ridge__alpha=417.53189365604004;; score=(train=-0.313, test=-0.309)
total time= 0.2s
[CV 4/5] END ridge__alpha=417.53189365604004;; score=(train=-0.317, test=-0.297)
total time= 0.2s
[CV 5/5] END ridge__alpha=417.53189365604004;; score=(train=-0.312, test=-0.315)
total time= 0.1s
[CV 1/5] END ridge__alpha=788.0462815669904;; score=(train=-0.319, test=-0.323) t
otal time= 0.2s
[CV 2/5] END ridge__alpha=788.0462815669904;; score=(train=-0.313, test=-0.344) t
otal time= 0.1s
[CV 3/5] END ridge__alpha=788.0462815669904;; score=(train=-0.321, test=-0.315) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=788.0462815669904;; score=(train=-0.324, test=-0.304) t
otal time= 0.1s
[CV 5/5] END ridge__alpha=788.0462815669904;; score=(train=-0.319, test=-0.323) t
otal time= 0.1s
[CV 1/5] END ridge__alpha=1487.3521072935118;; score=(train=-0.330, test=-0.333)
total time= 0.2s
[CV 2/5] END ridge__alpha=1487.3521072935118;; score=(train=-0.324, test=-0.353)
total time= 0.1s
[CV 3/5] END ridge__alpha=1487.3521072935118;; score=(train=-0.332, test=-0.326)
total time= 0.1s
[CV 4/5] END ridge__alpha=1487.3521072935118;; score=(train=-0.335, test=-0.315)
total time= 0.2s
[CV 5/5] END ridge__alpha=1487.3521072935118;; score=(train=-0.330, test=-0.335)
total time= 0.2s
[CV 1/5] END ridge__alpha=2807.2162039411755;; score=(train=-0.346, test=-0.347)
total time= 0.5s
[CV 2/5] END ridge__alpha=2807.2162039411755;; score=(train=-0.341, test=-0.367)
total time= 0.4s
[CV 3/5] END ridge__alpha=2807.2162039411755;; score=(train=-0.348, test=-0.341)
total time= 0.3s
[CV 4/5] END ridge__alpha=2807.2162039411755;; score=(train=-0.351, test=-0.331)
total time= 0.3s
[CV 5/5] END ridge__alpha=2807.2162039411755;; score=(train=-0.345, test=-0.353)
total time= 0.2s
[CV 1/5] END ridge__alpha=5298.316906283702;; score=(train=-0.366, test=-0.366) t
otal time= 0.2s
[CV 2/5] END ridge__alpha=5298.316906283702;; score=(train=-0.362, test=-0.386) t
otal time= 0.2s
[CV 3/5] END ridge__alpha=5298.316906283702;; score=(train=-0.368, test=-0.361) t
otal time= 0.2s
[CV 4/5] END ridge__alpha=5298.316906283702;; score=(train=-0.371, test=-0.352) t
otal time= 0.2s
[CV 5/5] END ridge__alpha=5298.316906283702;; score=(train=-0.365, test=-0.375) t
otal time= 0.2s
[CV 1/5] END ridge__alpha=10000.0;; score=(train=-0.389, test=-0.387) total time=
0.1s
[CV 2/5] END ridge__alpha=10000.0;; score=(train=-0.385, test=-0.407) total time=
0.1s
[CV 3/5] END ridge__alpha=10000.0;; score=(train=-0.391, test=-0.383) total time=
0.2s
[CV 4/5] END ridge__alpha=10000.0;; score=(train=-0.393, test=-0.375) total time=
0.1s
```

[CV 5/5] END ridge__alpha=10000.0;; score=(train=-0.387, test=-0.399) total time=0.2s
Done.

Best alpha : 0.0574

Best CV RMSE: 0.5336 (improvement over baseline: 0.1017)

```
In [14]: # — Extract per-fold RMSE for accurate error bands —————
results      = pd.DataFrame(grid_search.cv_results_)
alphas_tested = results['param_ridge__alpha'].astype(float).values

fold_train_rmse = np.array([
    np.sqrt(-results[f'split{k}_train_score'].values)
    for k in range(CV_FOLDS)
]) # shape (CV_FOLDS, n_alphas)

fold_val_rmse = np.array([
    np.sqrt(-results[f'split{k}_test_score'].values)
    for k in range(CV_FOLDS)
])

train_rmse      = fold_train_rmse.mean(axis=0)
train_rmse_std  = fold_train_rmse.std(axis=0)
cv_rmse         = fold_val_rmse.mean(axis=0)
cv_rmse_std     = fold_val_rmse.std(axis=0)

best_alpha = grid_search.best_params_['ridge__alpha']

# — Plot —————
fig, ax = plt.subplots(figsize=(11, 5))

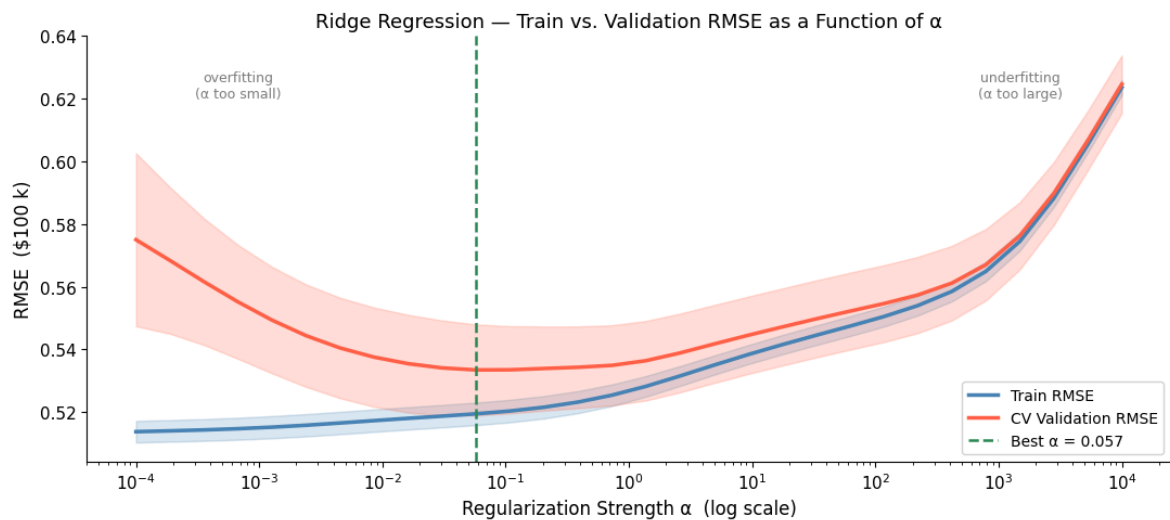
ax.semilogx(alphas_tested, train_rmse, color=COLORS[0], lw=2.5, label='Train RMS')
ax.fill_between(alphas_tested,
                train_rmse - train_rmse_std, train_rmse + train_rmse_std,
                alpha=0.2, color=COLORS[0])

ax.semilogx(alphas_tested, cv_rmse, color=COLORS[1], lw=2.5, label='CV Validation')
ax.fill_between(alphas_tested,
                cv_rmse - cv_rmse_std, cv_rmse + cv_rmse_std,
                alpha=0.2, color=COLORS[1])

ax.axvline(best_alpha, color=COLORS[2], linestyle='--', lw=1.8,
            label=f'Best  $\alpha$  = {best_alpha:.3f}')

# Annotate regions
ylo, yhi = ax.get_ylim()
ytext = ylo + (yhi - ylo) * 0.92
ax.text(alphas_tested[3], ytext, 'overfitting\n( $\alpha$  too small)',
        ha='center', va='top', fontsize=9, color='gray')
ax.text(alphas_tested[-4], ytext, 'underfitting\n( $\alpha$  too large)',
        ha='center', va='top', fontsize=9, color='gray')

ax.set_xlabel('Regularization Strength  $\alpha$  (log scale)', fontsize=12)
ax.set_ylabel('RMSE ($100 k)', fontsize=12)
ax.set_title('Ridge Regression – Train vs. Validation RMSE as a Function of  $\alpha$ ',
             fontsize=10)
ax.legend(fontsize=10)
plt.tight_layout()
plt.savefig('assets/ridge_validation_curve.png', dpi=130, bbox_inches='tight')
plt.show()
```



Interpretation:

- Left (small α): train RMSE low, CV RMSE high $\rightarrow$ overfitting
- Right (large α): both RMSEs high $\rightarrow$ underfitting
- Minimum of CV RMSE curve = best regularization strength

3.5 Model Evaluation on the Test Set

```
In [15]: best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f'Test RMSE (Ridge, degree-{POLY_DEGREE}): {test_rmse:.4f} ($100k)')
print(f'In dollar terms: ± ${test_rmse * 100_000:,.0f}')

# Residual plot
residuals = y_test - y_pred
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

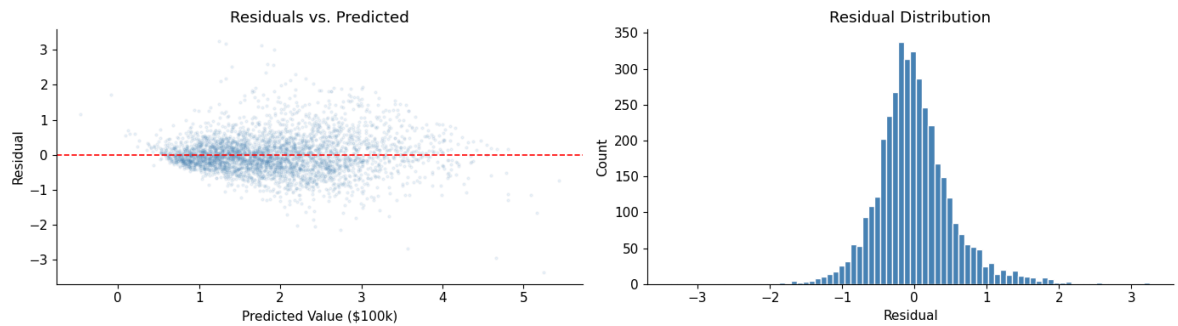
axes[0].scatter(y_pred, residuals, alpha=0.08, s=4, color=COLORS[0])
axes[0].axhline(0, color='red', lw=1.2, linestyle='--')
axes[0].set_xlabel('Predicted Value ($100k)')
axes[0].set_ylabel('Residual')
axes[0].set_title('Residuals vs. Predicted')

axes[1].hist(residuals, bins=80, color=COLORS[0], edgecolor='white')
axes[1].set_xlabel('Residual')
axes[1].set_ylabel('Count')
axes[1].set_title('Residual Distribution')

plt.tight_layout()
plt.savefig('assets/ridge_residuals.png', dpi=120, bbox_inches='tight')
plt.show()
```

Test RMSE (Ridge, degree-3): 0.5391 (\$100k)

In dollar terms: ± \$53,906



```
In [16]: bin_width_k = 50 # bin width in $k – try 25, 50, 100 to see coarser/finer break

pct_error = (y_pred - y_test) / y_test * 100 # signed: + = over-predict
abs_pct_error = np.abs(pct_error)

price_dollars = y_test * 100_000
bin_width = bin_width_k * 1_000
max_price = int(np.ceil(price_dollars.max() / bin_width) * bin_width) + bin_widt
bins = np.arange(0, max_price, bin_width)
labels = [f'${int(b/1000)}k-${int((b+bin_width)/1000)}k' for b in bins[:-1]]
price_bin = pd.cut(price_dollars, bins=bins, labels=labels)

bin_stats = pd.DataFrame({
    'abs_pct_error': abs_pct_error,
    'pct_error': pct_error,
    'price_bin': price_bin,
}).groupby('price_bin', observed=True).agg(
    median_abs_pct=('abs_pct_error', 'median'),
    p25=('abs_pct_error', lambda x: np.percentile(x, 25)),
    p75=('abs_pct_error', lambda x: np.percentile(x, 75)),
    n=('abs_pct_error', 'count'),
)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Left: scatter of signed % error vs actual price
axes[0].scatter(price_dollars / 1000, pct_error, alpha=0.05, s=3, color=COLORS[0])
axes[0].axhline(0, color='red', lw=1.2, linestyle='--')
axes[0].set_xlabel('Actual House Value ($k)')
axes[0].set_ylabel('Error (%) [(predicted - actual) / actual × 100]')
axes[0].set_title('Signed % Error vs. Actual Price')
axes[0].set_ylim(-100, 150)

# Right: binned median absolute % error with IQR band
x_pos = np.arange(len(bin_stats))
axes[1].bar(x_pos, bin_stats['median_abs_pct'], color=COLORS[0], alpha=0.8, label=
axes[1].vlines(x_pos,
                bin_stats['p25'], bin_stats['p75'],
                color='black', lw=2.5, label='IQR (25-75th pct)')
axes[1].scatter(x_pos, bin_stats['p75'], color='black', s=20, zorder=5)
axes[1].scatter(x_pos, bin_stats['p25'], color='black', s=20, zorder=5)

for i, (idx, row) in enumerate(bin_stats.iterrows()):
    axes[1].text(i, row['median_abs_pct'] + 0.5, f"{row['median_abs_pct']:.1f}%"
                 ha='center', va='bottom', fontsize=8)

axes[1].set_xticks(x_pos)
axes[1].set_xticklabels(bin_stats.index, rotation=40, ha='right', fontsize=8)
axes[1].set_xlabel('Actual Price Range')
```

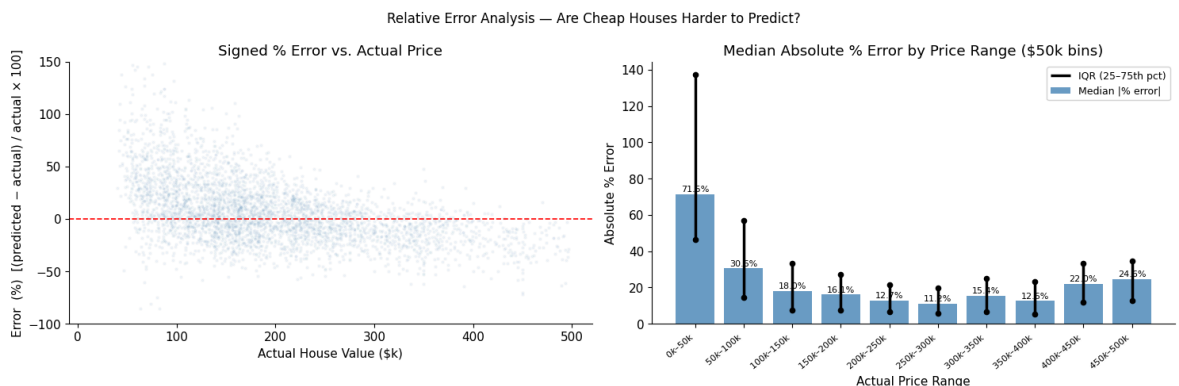
```

axes[1].set_ylabel('Absolute % Error')
axes[1].set_title(f'Median Absolute % Error by Price Range (${bin_width_k}k bins
axes[1].legend(fontsize=9)

plt.suptitle('Relative Error Analysis – Are Cheap Houses Harder to Predict?', fo
plt.tight_layout()
plt.savefig('assets/relative_error_by_price.png', dpi=130, bbox_inches='tight')
plt.show()

mape = abs_pct_error.mean()
print(f'MAPE (mean absolute % error): {mape:.1f}%')
print(f'Median absolute % error      : {abs_pct_error.median():.1f}%')

```



MAPE (mean absolute % error): 24.5%
Median absolute % error : 17.1%

```

In [17]: SMOOTH_WINDOW = 400 # number of nearest neighbors (by price) used for rolling q

# Sort test points by actual price so the rolling window is price-ordered
sort_idx = np.argsort(price_dollars.values)
price_s   = price_dollars.values[sort_idx] / 1_000 # in $k
pct_s     = pct_error.values[sort_idx]
abs_s     = abs_pct_error.values[sort_idx]

kw = dict(window=SMOOTH_WINDOW, center=True, min_periods=50)
med_signed = pd.Series(pct_s).rolling(**kw).median()
med_abs    = pd.Series(abs_s).rolling(**kw).median()
p25        = pd.Series(abs_s).rolling(**kw).quantile(0.25)
p75        = pd.Series(abs_s).rolling(**kw).quantile(0.75)

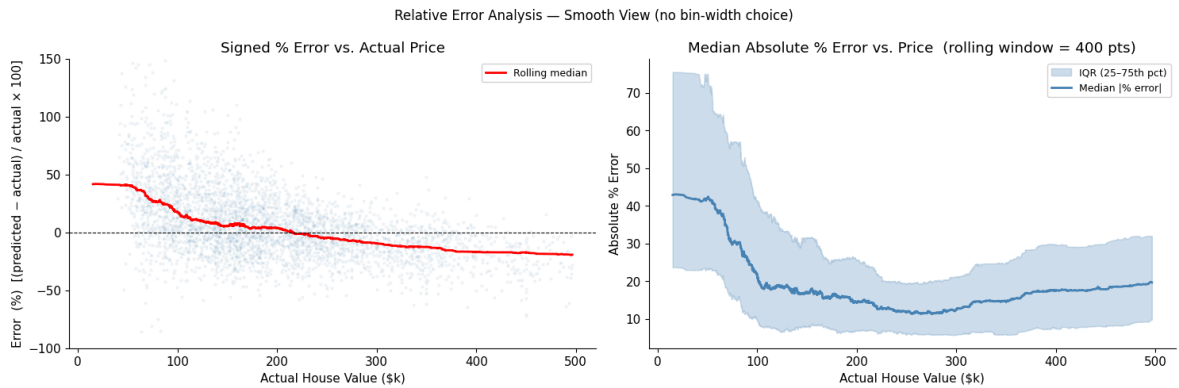
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Left: scatter + rolling median of signed % error
axes[0].scatter(price_s, pct_s, alpha=0.05, s=3, color=COLORS[0])
axes[0].plot(price_s, med_signed, color='red', lw=2, label='Rolling median')
axes[0].axhline(0, color='black', lw=0.8, linestyle='--')
axes[0].set_ylim(-100, 150)
axes[0].set_xlabel('Actual House Value ($k)')
axes[0].set_ylabel('Error (%) [(predicted - actual) / actual × 100]')
axes[0].set_title('Signed % Error vs. Actual Price')
axes[0].legend(fontsize=9)

# Right: rolling median absolute % error + IQR band
axes[1].fill_between(price_s, p25, p75, alpha=0.25, color=COLORS[0], label='IQR')
axes[1].plot(price_s, med_abs, color=COLORS[0], lw=2, label='Median [% error]')
axes[1].set_xlabel('Actual House Value ($k)')
axes[1].set_ylabel('Absolute % Error')
axes[1].set_title(f'Median Absolute % Error vs. Price (rolling window = {SMOOTH_WINDOW})')
axes[1].legend(fontsize=9)

```

```
plt.suptitle('Relative Error Analysis – Smooth View (no bin-width choice)', font
plt.tight_layout()
plt.savefig('assets/relative_error_smooth.png', dpi=130, bbox_inches='tight')
plt.show()
```



The smooth curve above shows the same signal as the bar chart, but without a bin-width choice. Both views are complementary: the bars give precise per-range numbers; the smooth curve makes the overall trend and local anomalies (the *300kbump*, *theriseabove400k*) easier to read at a glance.

What does this plot tell us?

The right panel reveals a clear pattern: a **steep monotonic decline** from the cheapest houses down to a sweet spot around 250k–300k, followed by a gradual rise at the expensive end.

Very cheap houses (0–50k) — 71.5% median error, IQR reaching ~140%

This is an extreme segment. Almost no California census blocks reported a median value below \$50k in 1990, so the model has barely any training signal here. The enormous IQR (the whisker reaches ~140%) tells us predictions are nearly random — the few data points are so heterogeneous that even their central tendency is uncertain.

The monotonic decline (50k– >250k) — 30.5% → 11.8%

Error falls steadily as house value increases. Two effects drive this:

1. *Data density.* The training distribution is centered around 150k–300k. As we move right across this range, the model sees progressively more examples and fits better.
2. *Percentage amplification.* A fixed absolute dollar error of, say, 15k is 30% of a 50k house but only 6% of a \$250k house. Even if prediction quality were identical across the range, the relative error metric would still show this downward slope.

Sweet spot (250k–300k) — 11.8% median error

This is where the model performs best: the highest concentration of training data, the most consistent income-location-price relationships, and the lowest percentage amplification effect. Absolute dollar errors in this bin are likely similar to neighboring bins — the relative metric just favors this range.

The slight bump at 300k–350k (15.4%) then recovery at 350k–400k (12.5%)

This small anomaly is worth noting. One plausible explanation is a feature interaction: blocks in the 300k–350k range may straddle a geographic boundary (e.g., the edge of a high-demand coastal county vs. an inland suburb) where the model's polynomial features produce a local misfit. With only 8 census variables, the model cannot distinguish otherwise-similar blocks that sit on opposite sides of a neighborhood boundary.

Expensive houses (400k–500k) — rising to 22–24.5%

Error climbs again at the high end for structural reasons:

1. *Proximity to the censoring boundary.* We removed the hard-capped \$500k rows, but blocks just below that cap may still carry artifacts of the censoring process — their features can be systematically unusual relative to blocks priced well below the ceiling.
2. *Luxury non-linearities.* High-end prices reflect factors the 8 census features cannot capture: school district prestige, architectural quality, micro-neighborhood cachet. The model undershoots these values.
3. *Sparse data.* Like the cheap end, few training examples exist here, so the model cannot specialize.

Left panel — signed errors

The scatter confirms the directional story: cheap houses are overwhelmingly **over-predicted** (positive signed error, large upward spread), while expensive houses cluster just below the zero line, reflecting systematic **under-prediction**. This is the classic **regression-to-the-mean** bias — a linear model trained on an asymmetric distribution anchors predictions toward the bulk of the data, dragging cheap predictions up and expensive predictions down.

Takeaway: relative (percentage) error is a more honest evaluation metric than RMSE for datasets with wide price ranges. RMSE would mask the poor performance at the extremes because those segments are numerically small and infrequent.

Exercise 1.1 — Effect of Polynomial Degree (Basic)

So far we used `POLY_DEGREE = 2`. In this exercise you will compare degree 1, 2, and 3 to see how polynomial degree interacts with regularization.

Tasks:

1. For each degree in `[1, 2, 3]`, build a Ridge pipeline, run `GridSearchCV` with the same `ALPHAS`, and record the best CV RMSE and the corresponding best α .
2. Print a summary table.
3. Answer: does a higher polynomial degree always improve test performance? Why or why not?

```

In [18]: DEGREES = [1, 2, 3]
degree_results = []
degree_best_estimators = {}

for degree in DEGREES:
    # Misma formulaci3n de preprocesamiento que en la Secci3n 3.3, solo cambia e
    log_preprocessor_deg = ColumnTransformer([
        ('log', FunctionTransformer(np.log1p, feature_names_out='one-to-one'),
        ('pass', 'passthrough', PASS_FEATURES),
    ], verbose_feature_names_out=False)

    pipeline_deg = Pipeline([
        ('log_transform', log_preprocessor_deg),
        ('poly', PolynomialFeatures(degree=degree, include_bias=False, interac
        ('scaler', StandardScaler()),
        ('ridge', Ridge()),
    ])

    grid_deg = GridSearchCV(
        pipeline_deg,
        {'ridge__alpha': ALPHAS},
        cv=CV_FOLDS,
        scoring='neg_mean_squared_error',
        n_jobs=-1,
    )
    grid_deg.fit(X_train, y_train)

    best_alpha_deg = grid_deg.best_params_['ridge__alpha']
    best_cv_rmse_deg = np.sqrt(-grid_deg.best_score_)

    y_pred_deg = grid_deg.best_estimator_.predict(X_test)
    test_rmse_deg = np.sqrt(mean_squared_error(y_test, y_pred_deg))
    n_features_deg = grid_deg.best_estimator_.named_steps['poly'].n_output_featu

    degree_best_estimators[degree] = grid_deg.best_estimator_
    degree_results.append({
        'degree': degree,
        'n_features': n_features_deg,
        'best_alpha': best_alpha_deg,
        'cv_rmse': best_cv_rmse_deg,
        'test_rmse': test_rmse_deg,
    })

    print(f'Degree {degree} ({n_features_deg:>3} features) '
          f'best  $\alpha$  = {best_alpha_deg:8.4f} | CV RMSE = {best_cv_rmse_deg:.4f}')

# — Tabla resumen —
results_df = pd.DataFrame(degree_results).set_index('degree')
print('\nSummary — Ridge CV performance by polynomial degree:')
display(results_df.round(4))

best_degree = results_df['cv_rmse'].idxmin()
print(f"\nBest CV RMSE achieved at degree = {best_degree}")

```

Degree 1 (8 features) best α = 2.5929 | CV RMSE = 0.6183 | Test RMSE = 0.6285
Degree 2 (44 features) best α = 0.0024 | CV RMSE = 0.5452 | Test RMSE = 0.5513
Degree 3 (164 features) best α = 0.0574 | CV RMSE = 0.5336 | Test RMSE = 0.5391

Summary – Ridge CV performance by polynomial degree:

	n_features	best_alpha	cv_rmse	test_rmse
degree				
1	8	2.5929	0.6183	0.6285
2	44	0.0024	0.5452	0.5513
3	164	0.0574	0.5336	0.5391

Best CV RMSE achieved at degree = 3

¿Un grado polinómico más alto siempre mejora el desempeño en test?

Con los números en la mano (Test RMSE: 0.6285 en grado 1, 0.5513 en grado 2 y 0.5391 en grado 3), la respuesta corta es NO!, no siempre, aunque en esta corrida en particular sí vimos una mejora en los tres casos.

Lo interesante no es que el error haya bajado, si no cuánto bajó cada vez. Al pasar de grado 1 a grado 2 el modelo gana mucho: el RMSE cae 0.077. Esto tiene sentido porque en grado 1 el modelo es puramente lineal, solo puede sumar cada variable multiplicada por un peso ($w_1 \cdot \text{MedInc} + w_2 \cdot \text{HouseAge} + \dots$), y no tiene forma de representar una curva. En grado 2 aparecen los términos cuadráticos y de interacción (MedInc^2 al cuadrado), $\text{MedInc} \times \text{Latitude}$, etc), que le dan al modelo piezas nuevas para armar justamente la relación no lineal entre ingreso y valor de la vivienda que vimos en la Sección 3.1. En cambio, al pasar de grado 2 a grado 3 el número de features casi se cuadruplica (44 -> 164) y sin embargo el RMSE apenas mejora 0.012. Es una mejora real, pero minúscula comparada con el costo de complejidad que se pagó para conseguirla.

Esto es el compromiso sesgo–varianza en acción: cada grado adicional le da al modelo más libertad para ajustarse a los datos de entrenamiento (menos sesgo), pero también más oportunidades de capturar ruido en vez de señal (más varianza). `GridSearchCV` se defiende de esa varianza extra subiendo la regularización, el mejor α pasa de 2.59 en grado 1 a valores mucho más pequeños (0.0024 y 0.057) en grados 2 y 3, pero esa defensa tiene un límite: no puede convertir features irrelevantes en información útil, solo puede amortiguar el daño.

Por eso no conviene generalizar la conclusión a "más grado, mejor modelo". Lo que muestran estos resultados es más bien un rendimiento decreciente: en algún punto (quizá ya en el grado 3, quizá un poco más adelante) seguir subiendo el grado dejaría de aportar y empezaría a jugar en contra, con un modelo más lento de entrenar, más difícil de interpretar y más propenso a sobreajustar. La forma correcta de decidir el grado no es intuición ni "mientras más mejor", sino justamente lo que hicimos aquí: comparar el error de validación cruzada entre varias opciones y quedarse con la que realmente lo minimiza.

Exercise 1.2 — Ridge vs. Lasso

Ridge (L2) shrinks all coefficients toward zero but rarely to exactly zero.

Lasso (L1) tends to produce **sparse** models — many coefficients become exactly zero, effectively performing feature selection.

Tasks:

1. Build a Lasso pipeline (same structure: `PolynomialFeatures` → `StandardScaler` → `Lasso`) and tune `lasso__alpha` with `GridSearchCV`.
2. Compare test RMSE for Ridge and Lasso.
3. Count how many of the degree-2 polynomial coefficients are exactly zero for each model. What does this tell you about feature selection?

4. Plot the top-20 non-zero Lasso coefficients.

```
In [20]: POLY_DEGREE_EX12 = 2 # el enunciado pide comparar los coeficientes de un polino

# Mismo preprocesamiento (log1p + passthrough) que en La Sección 3.3 / Ejercicio
log_preprocessor_ex12 = ColumnTransformer([
    ('log', FunctionTransformer(np.log1p, feature_names_out='one-to-one'), LOG_
    ('pass', 'passthrough', PASS_FEATURES),
], verbose_feature_names_out=False)

# — Pipeline Ridge (mismo grado que Lasso, para comparar en igualdad de condici
ridge_pipeline_ex12 = Pipeline([
    ('log_transform', log_preprocessor_ex12),
    ('poly', PolynomialFeatures(degree=POLY_DEGREE_EX12, include_bias=False)),
    ('scaler', StandardScaler()),
    ('ridge', Ridge()),
])

ridge_grid_ex12 = GridSearchCV(
    ridge_pipeline_ex12,
    {'ridge__alpha': ALPHAS},
    cv=CV_FOLDS,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
)
ridge_grid_ex12.fit(X_train, y_train)

# — Pipeline Lasso (misma estructura, cambia solo el estimador final) —
lasso_pipeline_ex12 = Pipeline([
    ('log_transform', log_preprocessor_ex12),
    ('poly', PolynomialFeatures(degree=POLY_DEGREE_EX12, include_bias=False)),
    ('scaler', StandardScaler()),
    ('lasso', Lasso(max_iter=20_000)),
])

lasso_grid_ex12 = GridSearchCV(
    lasso_pipeline_ex12,
    {'lasso__alpha': ALPHAS},
    cv=CV_FOLDS,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
)
lasso_grid_ex12.fit(X_train, y_train)

ridge_best_ex12 = ridge_grid_ex12.best_estimator_
lasso_best_ex12 = lasso_grid_ex12.best_estimator_

# — Tarea 2: comparar Test RMSE —
ridge_test_rmse_ex12 = np.sqrt(mean_squared_error(y_test, ridge_best_ex12.predict_
lasso_test_rmse_ex12 = np.sqrt(mean_squared_error(y_test, lasso_best_ex12.predict_

print(f'Ridge (grado {POLY_DEGREE_EX12}) → best α = {ridge_grid_ex12.best_para
print(f'Lasso (grado {POLY_DEGREE_EX12}) → best α = {lasso_grid_ex12.best_para

# — Tarea 3: contar coeficientes exactamente en cero —
ridge_coefs_ex12 = ridge_best_ex12.named_steps['ridge'].coef_
lasso_coefs_ex12 = lasso_best_ex12.named_steps['lasso'].coef_
n_total_coefs_ex12 = len(lasso_coefs_ex12)
```

```

n_ridge_zero_ex12 = int(np.sum(ridge_coefs_ex12 == 0))
n_lasso_zero_ex12 = int(np.sum(lasso_coefs_ex12 == 0))

print(f'\nTotal de coeficientes (features de grado {POLY_DEGREE_EX12}): {n_total}')
print(f'Ridge → coeficientes exactamente en cero: {n_ridge_zero_ex12} / {n_total}')
print(f'Lasso → coeficientes exactamente en cero: {n_lasso_zero_ex12} / {n_total}')
print(f'({n_lasso_zero_ex12} / {n_total:.1%})')

# — Tarea 4: top-20 coeficientes no nulos de Lasso —————
# El ColumnTransformer produce las columnas en este orden: log-transformadas + p
input_feature_names_ex12 = LOG_FEATURES + PASS_FEATURES
feature_names_ex12 = lasso_best_ex12.named_steps['poly'].get_feature_names_out(input_feature_names_ex12)

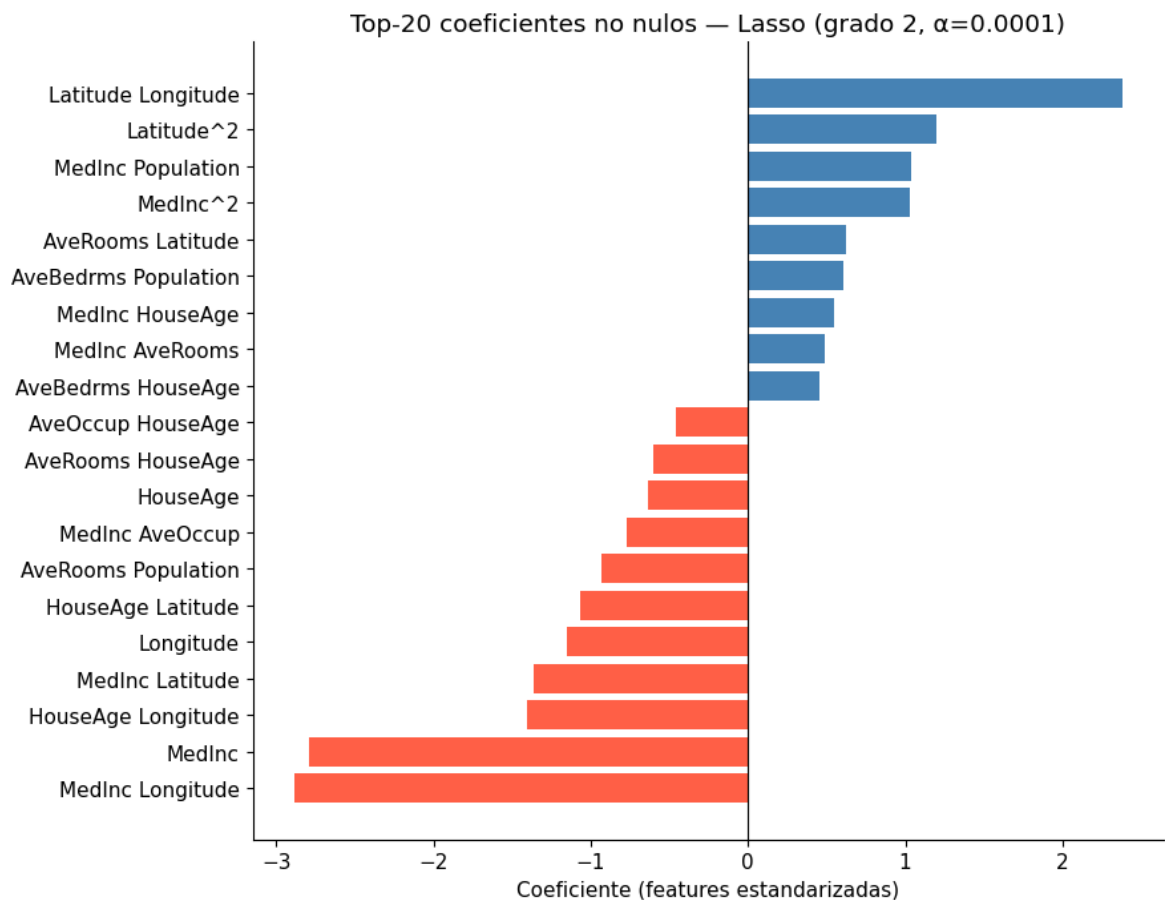
lasso_coef_df_ex12 = pd.DataFrame({
    'feature': feature_names_ex12,
    'coef': lasso_coefs_ex12,
})
lasso_nonzero_ex12 = lasso_coef_df_ex12[lasso_coef_df_ex12['coef'] != 0].copy()
lasso_nonzero_ex12['abs_coef'] = lasso_nonzero_ex12['coef'].abs()
top20_ex12 = lasso_nonzero_ex12.sort_values('abs_coef', ascending=False).head(20)

fig, ax = plt.subplots(figsize=(9, 7))
bar_colors_ex12 = [COLORS[1] if c < 0 else COLORS[0] for c in top20_ex12['coef']]
ax.barh(top20_ex12['feature'], top20_ex12['coef'], color=bar_colors_ex12)
ax.axvline(0, color='black', lw=0.8)
ax.set_xlabel('Coeficiente (features estandarizadas)')
ax.set_title(f'Top-20 coeficientes no nulos – Lasso (grado {POLY_DEGREE_EX12}, '
             f'α={lasso_grid_ex12.best_params_["lasso__alpha"]:.4f})')
plt.tight_layout()
plt.savefig('assets/lasso_top20_coefficients.png', dpi=120, bbox_inches='tight')
plt.show()

```

Ridge (grado 2) → best α = 0.0024 | Test RMSE = 0.5513
 Lasso (grado 2) → best α = 0.0001 | Test RMSE = 0.5634

Total de coeficientes (features de grado 2): 44
 Ridge → coeficientes exactamente en cero: 0 / 44
 Lasso → coeficientes exactamente en cero: 9 / 44 (20.5%)



¿Qué nos dice esto sobre la selección de features?

Con Test RMSE de 0.5513 (Ridge) vs 0.5634 (Lasso), Ridge generaliza un poco mejor en este caso, tiene sentido, porque ninguna de sus 44 features es realmente "inútil": todas aportan algo de señal, aunque sea pequeño y penalizarlas con $\alpha \cdot \|w\|^2$ en vez de eliminarlas aprovecha mejor esa información. Lasso en cambio, apaga por completo 9 de los 44 coeficientes (20.5%), su penalización $\alpha \cdot \|w\|_1$ tiene una "esquina" en cero que hace que para features poco informativas, sea más barato llevarlas exactamente a 0 que dejarlas con un valor pequeño.

Eso es justamente lo que convierte a Lasso en un método de selección automática de features: en vez de usar las 44 columnas que genera `PolynomialFeatures(degree=2)` (8 originales + cuadrados + interacciones), el modelo se queda solo con 35 y descarta el resto sin que nosotros tengamos que decidirlo a mano. El gráfico de arriba muestra cuáles sobrevivieron con más peso: `Latitude × Longitude` (la ubicación geográfica combinada) es el término más importante, seguido de `Latitude`(al cuadrado) y de interacciones con `MedInc` (`MedInc × Population`, `MedInc`(al cuadrado), `MedInc × HouseAge`). Esto confirma algo intuitivo sobre el mercado inmobiliario: el precio no depende solo del ingreso o de la ubicación por separado, sino de cómo interactúan, la misma renta vale mucho más en una zona que en otra.

Por otro lado, debemos destacar que esa poda no es gratis: si dos features están muy correlacionadas, Lasso tiende a quedarse con una y descartar la otra casi arbitrariamente, mientras que Ridge reparte el peso entre ambas de forma más estable, probablemente por eso Ridge termina con menor error en test aquí. En definitiva: si lo que buscamos es interpretabilidad y saber qué variables importan, Lasso es más útil y si lo que importa es

la estabilidad del ajuste y exprimir el último punto de desempeño, Ridge sigue siendo la opción más segura.

4. Part 2 — Online Learning with SGDRegressor

4.1 When Data Doesn't Fit in Memory

The California Housing dataset fits comfortably in RAM. But in the real world, datasets are often large enough to exceed available memory:

- **Streaming music platforms** accumulate audio features for millions of new songs every year.
- **E-commerce sites** log billions of user interactions monthly.
- **Medical imaging pipelines** generate terabytes of feature matrices.

For these scenarios, loading all data before training is impossible. The solution is **online (incremental) learning**: process one mini-batch at a time, update model weights, discard the batch, and move on — without ever holding the full dataset in RAM.

```
In [21]: # Memory footprint estimate for the MSD dataset
n_train  = MSD_TRAIN_ROWS
n_test   = MSD_TEST_ROWS
n_feat   = 90
bytes_f64 = 8

raw_mb = (n_train + n_test) * (n_feat + 1) * bytes_f64 / 1024**2
print(f'Raw feature matrix ({n_train + n_test:,} rows x {n_feat} features): ~{ra

# Degree-2 polynomial features would be huge
n_poly2 = PolynomialFeatures(degree=2, include_bias=False).fit(np.zeros((1, n_fe
poly2_gb = (n_train + n_test) * n_poly2 * bytes_f64 / 1024**3
print(f'After degree-2 expansion ({n_poly2:,} features): ~{poly2_gb:.0f} GB ← i
```

Raw feature matrix (515,345 rows × 90 features): ~358 MB

After degree-2 expansion (4,185 features): ~16 GB ← impossible to load at once

Even the raw matrix for a full year (1 M songs) would be ~750 MB. With feature engineering and multiple frames it quickly exceeds available RAM. Solution: stream data in chunks and use `partial_fit` for online learning.

4.2 Dataset: Year Prediction MSD

The **Million Song Dataset (MSD)** is a collection of audio features and metadata for one million contemporary pop music tracks. This lab uses a preprocessed subset from the [UCI Machine Learning Repository](#).

Property	Value
Training samples	515,345

Property	Value
Test samples	51,630
Features	90 (timbre-based audio)
Target	Song release year (1922 – 2011)
File size	~700 MB unzipped

Feature description:

Features 1–12 are the **timbre average**: the mean of each of 12 **timbre** components extracted by the Echo Nest API. Timbre captures the texture and tone quality of a sound — roughly what makes a piano sound different from a violin even at the same pitch and loudness.

Features 13–90 are the **timbre covariance**: the 78 upper-triangle values of the 12×12 covariance matrix, capturing how the timbre components vary together across the song.

Task: Predict the year a song was released from these audio statistics.

Important: The train/test split is **pre-defined** by the dataset creators (first 515,345 rows = train, last 51,630 = test). Do not shuffle the file; use this fixed split to ensure comparable results.

```
In [22]: def _reporhook(count, block_size, total_size):
    pct = min(100, count * block_size * 100 // total_size) if total_size > 0 else
    mb = count * block_size / 1024**2
    print(f'\r {pct:3d}% ({mb:.0f} MB downloaded)', end='', flush=True)

    if not os.path.exists(MSD_DATA_FILE):
        zip_file = 'YearPredictionMSD.txt.zip'
        if not os.path.exists(zip_file):
            print(f'Downloading {MSD_URL} ...')
            print(f'(~200 MB compressed; may take a few minutes on Colab)')
            urllib.request.urlretrieve(MSD_URL, zip_file, reporthook=_reporhook)
            print()
            print('Extracting ...')
            with zipfile.ZipFile(zip_file, 'r') as z:
                z.extractall('.')
            size_mb = os.path.getsize(MSD_DATA_FILE) / 1024**2
            print(f'Done. {MSD_DATA_FILE} ({size_mb:.0f} MB)')
        else:
            size_mb = os.path.getsize(MSD_DATA_FILE) / 1024**2
            print(f'Dataset already present: {MSD_DATA_FILE} ({size_mb:.0f} MB)')
```

Downloading <https://archive.ics.uci.edu/ml/machine-learning-databases/00203/YearPredictionMSD.txt.zip> ...

(~200 MB compressed; may take a few minutes on Colab)

0% (201 MB downloaded)

Extracting ...

Done. YearPredictionMSD.txt (428 MB)

```
In [23]: # Quick EDA – read a small slice so we don't load the full file
df_eda = pd.read_csv(MSD_DATA_FILE, header=None, nrows=10_000)
feature_cols = ['year'] + [f'f{i}' for i in range(1, 91)]
df_eda.columns = feature_cols
```

```

print(f'Sample shape: {df_eda.shape}')
print(f'Year range : {df_eda["year"].min()} - {df_eda["year"].max()}')
print(f'Year mean  : {df_eda["year"].mean():.1f}')
print(f'Year std   : {df_eda["year"].std():.1f}')

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

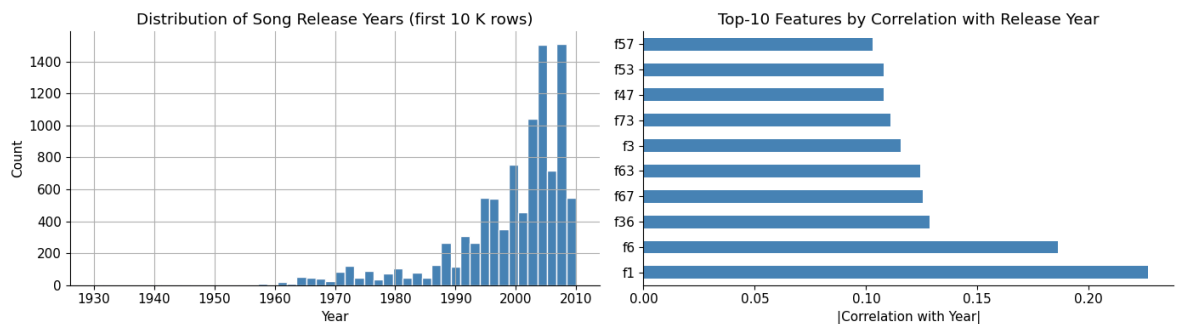
df_eda['year'].hist(bins=50, ax=axes[0], color=COLORS[0], edgecolor='white')
axes[0].set_xlabel('Year')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Song Release Years (first 10 K rows)')

corrs = df_eda.corr()['year'].drop('year').abs().sort_values(ascending=False).head(10)
corrs.plot(kind='barh', ax=axes[1], color=COLORS[0])
axes[1].set_xlabel('|Correlation with Year|')
axes[1].set_title('Top-10 Features by Correlation with Release Year')

plt.tight_layout()
plt.savefig('assets/msd_eda.png', dpi=120, bbox_inches='tight')
plt.show()

```

Sample shape: (10000, 91)
 Year range : 1930 - 2010
 Year mean : 1998.9
 Year std : 10.2



4.3 SGDRegressor and partial_fit

From Batch GD to Mini-Batch SGD

Batch gradient descent computes the gradient over *all* N training samples before each weight update:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

This is exact but requires all data in memory and is slow for large N.

Stochastic gradient descent (SGD) uses a single random sample per update:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

Mini-batch SGD strikes a balance — use a small batch of B samples:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

This is both memory-efficient and statistically stable.

`partial_fit` in `sklearn`

`SGDRegressor.partial_fit(X_batch, y_batch)` applies **one pass** of mini-batch SGD on the provided batch without forgetting previously learned weights. Calling it repeatedly on successive batches (or even on multiple epochs over the same data) implements full online/incremental learning.

```
sgd = SGDRegressor(learning_rate='constant', eta0=0.01)
```

```
for chunk in pd.read_csv('bigfile.csv', chunksize=50_000):
    X_chunk = chunk.iloc[:, 1:].values
    y_chunk = chunk.iloc[:, 0].values
    sgd.partial_fit(X_scaled, y_chunk)    # weights updated in-place
```

Key rule: the scaler must be fitted on **training data only** — never re-fitted on test data or future production chunks — then applied (`transform`) consistently everywhere. The cleanest approach (used in Exercise 2.1) is a dedicated first pass over **all** training chunks with `StandardScaler.partial_fit`, which gives accurate statistics regardless of dataset size. For datasets where two passes are prohibitively expensive, fitting on a **representative sample** (e.g. the first few chunks) is a common practical shortcut, provided the sample reflects the overall distribution.

Exercise 2.1 — Implement the Mini-Batch Training Loop (Basic)

Implement one epoch of mini-batch SGD over the MSD training data using `SGDRegressor.partial_fit`.

Tasks:

1. Create an `SGDRegressor` with `loss='squared_error'`, `learning_rate='constant'`, `eta0=0.001`, `random_state=42`.
2. Create a `StandardScaler`.
3. **Pass 1 — fit the scaler:** stream through the training rows and call `msd_scaler.partial_fit(X_chunk)` on every chunk so the scaler captures the full training distribution.
4. **Pass 2 — train:** stream through the data again. For each chunk:
 - Extract `y_chunk` (column 0) and `X_chunk` (columns 1–90).
 - Transform `X_chunk` with the already-fitted scaler.
 - Call `sgd.partial_fit(X_scaled, y_chunk)`.
 - Compute RMSE on the current chunk (after the update) and store it in `chunk_rmse`.
5. Print the total number of chunks processed and the final chunk RMSE.

Practical note on scaler fitting: Pass 1 here scans *all* training chunks with `partial_fit`, giving the most accurate mean and variance estimates —

ideal when two passes are affordable. For very large datasets where two passes are too costly, fitting on a **representative sample** (e.g. the first chunk or a random subset) is an acceptable approximation, provided the sample is reasonably representative of the full distribution. In production streaming systems where the data distribution can shift over time (**concept drift**), consider periodically re-calling `partial_fit` on a recent window of data to keep the scaler's statistics fresh.

Expected output: ~10–11 chunks processed, final chunk RMSE around 8–11 years.

```
In [25]: sgd = SGDRegressor(loss='squared_error', learning_rate='constant', eta0=0.001, r
msd_scaler = StandardScaler()

# — Pasada 1: ajustar el scaler recorriendo todos los chunks de entrenamiento —
train_reader_pass1 = pd.read_csv(MSD_DATA_FILE, header=None, chunksize=CHUNK_SIZ
for chunk in train_reader_pass1:
    X_chunk = chunk.iloc[:, 1:].values
    msd_scaler.partial_fit(X_chunk)

print(f'Scaler ajustado sobre {MSD_TRAIN_ROWS:,} filas de entrenamiento (todas l

# — Pasada 2: entrenar SGD chunk a chunk con partial_fit —————
chunk_rmse = []
train_reader_pass2 = pd.read_csv(MSD_DATA_FILE, header=None, chunksize=CHUNK_SIZ
for chunk in train_reader_pass2:
    y_chunk = chunk.iloc[:, 0].values
    X_chunk = chunk.iloc[:, 1:].values
    X_scaled = msd_scaler.transform(X_chunk)

    sgd.partial_fit(X_scaled, y_chunk)

    # RMSE del chunk actual evaluado justo después de la actualización de pesos
    y_pred_chunk = sgd.predict(X_scaled)
    chunk_rmse = np.sqrt(mean_squared_error(y_chunk, y_pred_chunk))
    chunk_rmse.append(chunk_rmse)

print(f'\nChunks procesados : {len(chunk_rmse)}')
print(f'RMSE del último chunk: {chunk_rmse[-1]:.4f} años')
```

Scaler ajustado sobre 463,715 filas de entrenamiento (todas las columnas 1–90).

Chunks procesados : 93

RMSE del último chunk: 9.7518 años

Exercise 2.2 — Plot the Convergence Curve (Basic)

Using the `chunk_rmse` list from Exercise 2.1, plot the RMSE as a function of chunk number to visualise convergence within one epoch.

Tasks:

1. Plot RMSE vs. chunk index.
2. Add axis labels and a title.

3. Answer: does the model converge within a single pass? What pattern do you observe?

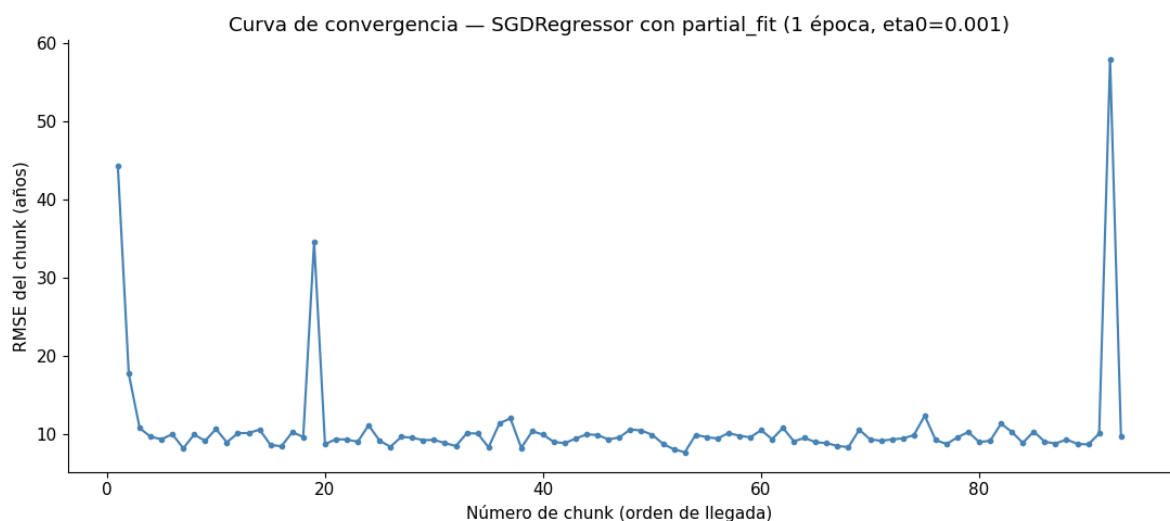
```
In [26]: fig, ax = plt.subplots(figsize=(11, 5))

chunk_idx = np.arange(1, len(chunk_rmses) + 1)
ax.plot(chunk_idx, chunk_rmses, color=COLORS[0], marker='o', markersize=3, lw=1.5)

ax.set_xlabel('Número de chunk (orden de llegada)')
ax.set_ylabel('RMSE del chunk (años)')
ax.set_title('Curva de convergencia — SGDRegressor con partial_fit (1 época, eta0=0.001)')

plt.tight_layout()
plt.savefig('assets/sgd_convergence_curve.png', dpi=120, bbox_inches='tight')
plt.show()

print(f'RMSE primer chunk : {chunk_rmses[0]:.4f} años')
print(f'RMSE último chunk : {chunk_rmses[-1]:.4f} años')
print(f'RMSE mínimo      : {min(chunk_rmses):.4f} años (chunk {int(np.argmin(chunk_rmses))})')
```



RMSE primer chunk : 44.2414 años
RMSE último chunk : 9.7518 años
RMSE mínimo : 7.7227 años (chunk 53)

¿El modelo converge dentro de una sola pasada? ¿Qué patrón se observa?

Sí, converge y lo hace muy rápido. El primer chunk arranca con un RMSE de 44.2414 años (los pesos parten en cero, así que las primeras predicciones son prácticamente una adivinanza), pero para el segundo chunk ya cae a 17.8, y para el chunk 4-5 el error ya se estabiliza alrededor de 8-11 años. El resto de la época (unos 90 chunks) se mueve dentro de esa misma banda, sin una tendencia clara a seguir bajando: el modelo ya extrajo casi toda la señal lineal que este eta0 le permite en una sola pasada.

Lo más llamativo del gráfico son dos picos puntuales, uno cerca del chunk 19 (RMSE $\approx$ 34.6) y otro cerca del chunk 92 (RMSE $\approx$ 58.0), que rompen momentáneamente esa estabilidad. La explicación más probable es que el archivo no está barajado: los datos del Million Song Dataset vienen ordenados según algún criterio propio del dataset original (por ejemplo, agrupados por artista o por año de forma parcial), así que un chunk de 5,000 filas consecutivas puede caer, por azar, en un tramo poco representativo de la distribución general (por ejemplo, muchas canciones de una década o un estilo musical

inusual). Como el modelo nunca vio nada parecido, sus pesos actuales fallan fuerte en ese chunk puntual, pero apenas se actualiza con ese lote y pasa al siguiente, el error vuelve a caer a la banda normal, porque el resto de los datos sigue siendo consistente con lo aprendido hasta ese momento.

Esto es justamente el motivo por el que el Ejercicio 2.3 introduce el shuffle de los datos al inicio de cada época: si cada chunk fuera una muestra aleatoria de todo el dataset (en vez de un bloque contiguo del archivo), estos picos por "chunks atípicos" prácticamente desaparecerían, porque cada lote sería representativo de la distribución completa.

Exercise 2.3 — Multi-Epoch Training (Intermediate)

One pass through the data is often insufficient. Run `N_EPOCHS` epochs and track **validation RMSE** on the held-out test set after each epoch.

Tasks:

1. Load the test set: `pd.read_csv(MSD_DATA_FILE, header=None, skiprows=MSD_TRAIN_ROWS)`. Scale `X_test` using the **already-fitted** scaler from Exercise 2.1 (`msd_scaler`).
2. Train a fresh `SGDRegressor` (same settings as 2.1) for `N_EPOCHS` epochs, recording both per-chunk train RMSE and per-epoch validation RMSE. **Shuffle the training data at the start of each epoch** so the model sees a different ordering every pass.
3. Plot train RMSE per chunk (all epochs concatenated) and validation RMSE per epoch on the same figure.
4. Answer: does validation RMSE improve across epochs? When does it saturate?

```
In [27]: # — Tarea 1: cargar el test set y escalarlo con el scaler ya ajustado en el Eje
test_df_ex23 = pd.read_csv(MSD_DATA_FILE, header=None, skiprows=MSD_TRAIN_ROWS)
y_test_msd = test_df_ex23.iloc[:, 0].values
X_test_msd = test_df_ex23.iloc[:, 1:].values
X_test_msd_scaled = msd_scaler.transform(X_test_msd) # solo transform, el scale

print(f'Test set cargado : {X_test_msd.shape[0]:,} filas')

# Cargamos también todo el train set en memoria una sola vez, para poder barajar
# el orden de las filas al inicio de cada época (no es posible barajar un
# iterador de pd.read_csv sin releer el archivo completo en cada época).
train_df_ex23 = pd.read_csv(MSD_DATA_FILE, header=None, nrows=MSD_TRAIN_ROWS)
y_train_msd = train_df_ex23.iloc[:, 0].values
X_train_msd = train_df_ex23.iloc[:, 1:].values
n_train_msd = X_train_msd.shape[0]

print(f'Train set cargado: {n_train_msd:,} filas')

# — Tarea 2: entrenar un SGDRegressor nuevo durante N_EPOCHS, barajando cada ép
sgd_multi = SGDRegressor(loss='squared_error', learning_rate='constant', eta0=0.
rng_ex23 = np.random.default_rng(RANDOM_STATE)

train_chunk_rmses_multi = [] # RMSE por chunk, con todas las épocas concatenad
```

```

val_rmses_per_epoch = []          # RMSE de validación, uno por época

for epoch in range(N_EPOCHS):
    shuffled_idx = rng_ex23.permutation(n_train_msd) # nuevo orden aleatorio en

    for start in range(0, n_train_msd, CHUNK_SIZE):
        batch_idx = shuffled_idx[start:start + CHUNK_SIZE]
        X_chunk = msd_scaler.transform(X_train_msd[batch_idx])
        y_chunk = y_train_msd[batch_idx]

        sgd_multi.partial_fit(X_chunk, y_chunk)

        chunk_rmse = np.sqrt(mean_squared_error(y_chunk, sgd_multi.predict(X_chunk)))
        train_chunk_rmses_multi.append(chunk_rmse)

    y_val_pred = sgd_multi.predict(X_test_msd_scaled)
    val_rmse = np.sqrt(mean_squared_error(y_test_msd, y_val_pred))
    val_rmses_per_epoch.append(val_rmse)

    print(f'Época {epoch + 1:>2}/{N_EPOCHS} → Val RMSE = {val_rmse:.4f} años')

# — Tarea 3: graficar train RMSE por chunk (todas las épocas) y val RMSE por época
n_chunks_per_epoch = int(np.ceil(n_train_msd / CHUNK_SIZE))
epoch_end_positions = [(e + 1) * n_chunks_per_epoch for e in range(N_EPOCHS)]

fig, ax = plt.subplots(figsize=(12, 5))

chunk_idx_multi = np.arange(1, len(train_chunk_rmses_multi) + 1)
ax.plot(chunk_idx_multi, train_chunk_rmses_multi, color=COLORS[0], lw=1, alpha=0.5,
        label='Train RMSE (por chunk)')
ax.plot(epoch_end_positions, val_rmses_per_epoch, color=COLORS[1], marker='o', lw=1,
        label='Val RMSE (por época)')

for epoch_end in epoch_end_positions[:-1]:
    ax.axvline(epoch_end, color='gray', lw=0.5, linestyle=':', alpha=0.5)

ax.set_xlabel('Chunk (todas las épocas concatenadas)')
ax.set_ylabel('RMSE (años)')
ax.set_title(f'Entrenamiento multi-época — SGDRegressor ({N_EPOCHS} épocas, para {N_CHUNKS} chunks)')
ax.legend()
plt.tight_layout()
plt.savefig('assets/sgd_multiepoch_convergence.png', dpi=120, bbox_inches='tight')
plt.show()

```

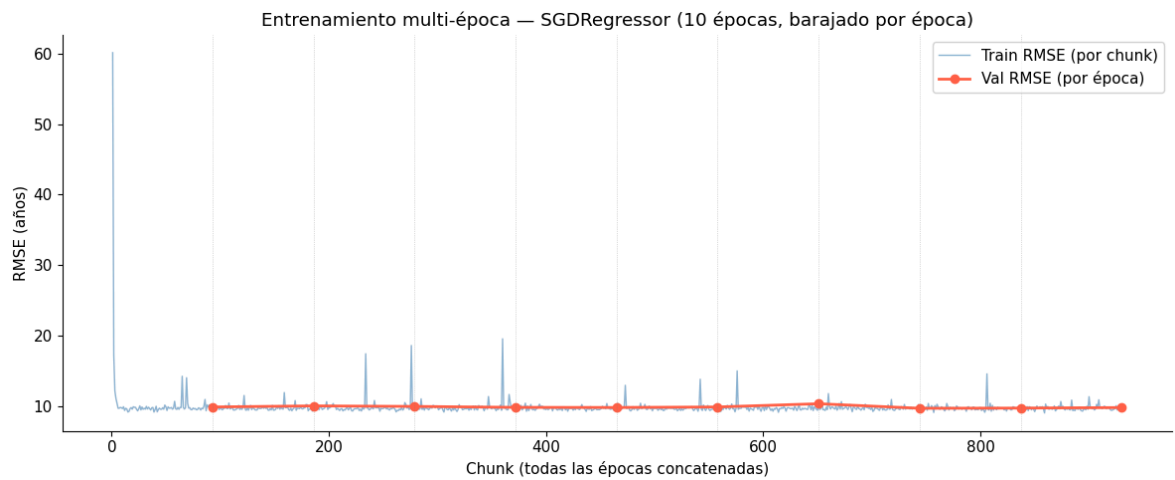
Test set cargado : 51,630 filas

Train set cargado: 463,715 filas

```

Época 1/10 → Val RMSE = 9.9274 años
Época 2/10 → Val RMSE = 10.0818 años
Época 3/10 → Val RMSE = 10.0053 años
Época 4/10 → Val RMSE = 9.8727 años
Época 5/10 → Val RMSE = 9.8559 años
Época 6/10 → Val RMSE = 9.9298 años
Época 7/10 → Val RMSE = 10.3874 años
Época 8/10 → Val RMSE = 9.7426 años
Época 9/10 → Val RMSE = 9.7712 años
Época 10/10 → Val RMSE = 9.8436 años

```



¿La validación mejora entre épocas? ¿Cuándo satura?

Prácticamente no. El RMSE de validación por época se mueve así: 9.93 -> 10.08 -> 10.01 -> 9.87 -> 9.86 -> 9.93 -> 10.39 -> 9.74 -> 9.77 -> 9.84 años. No hay una tendencia sostenida hacia abajo: los valores oscilan dentro de una banda de aproximadamente 9.7–10.4 años desde la primera época, con la época 7 como un pico aislado (10.39) y la época 8 como el mejor resultado (9.74), una diferencia tan pequeña entre el mínimo y el máximo que bien podría deberse al ruido normal de SGD y no a una mejora real del modelo.

En otras palabras, el modelo ya saturó en la época 1: la primera pasada completa por los datos (con η_0 constante) alcanza casi de inmediato la región de error que el modelo es capaz de lograr con esta configuración, y las nueve épocas siguientes no aportan una mejora consistente, solo hacen que el modelo "dé vueltas" alrededor del mismo nivel de error. Esto es coherente con lo que vimos en el Ejercicio 2.2: dentro de una sola época el error del train ya caía a la banda estable de ~9-10 años en los primeros chunks, así que darle más épocas al modelo no le da información nueva, solo se la repite (ahora barajada, así que sin los picos puntuales del Ejercicio 2.2).

Esto también expone una limitación de usar una tasa de aprendizaje constante: como η_0 no decae, el modelo nunca deja de dar "pasos grandes" en cada actualización, así que en vez de ir afinando el ajuste época a época, se queda rebotando alrededor del mínimo sin asentarse en él con precisión. Ese es justamente el problema que exploran los Ejercicios 2.4 y 2.5: qué pasa con learning rates distintos y con schedules que sí decaen con el tiempo.

Exercise 2.4 — Learning Rate Exploration (Key Exercise)

The learning rate `eta0` is the most sensitive hyperparameter in SGD. Too large, and gradient updates overshoot the minimum, causing **divergence**. Too small, and learning is too slow to be practical.

Tasks:

1. Train five separate `SGDRegressor` models, each with a different `eta0` from `[0.0001, 0.001, 0.01, 0.1, 1.0]`, each for **one epoch** over the training data.
2. For each model, track per-chunk RMSE. If RMSE becomes `NaN` or `inf`, stop training that model early and mark it as diverged.
3. Plot all five convergence curves on the same axes. Cap the y-axis at 150 years so diverging runs don't dominate.
4. Answer the following:
 - Which learning rates diverge?
 - Why does a large learning rate cause divergence? Sketch the geometry.
 - What is the cost of using too small a learning rate?

```
In [30]: ETA0_VALUES = [0.0001, 0.001, 0.01, 0.1, 1.0]
DIVERGENCE_THRESHOLD = 10_000 # años: muy por encima del rango de warm-up normal

# Reutiliza X_train_msd / y_train_msd (ya cargados en memoria en el Ejercicio 2.
# y msd_scaler (ya ajustado en el Ejercicio 2.1); una sola época, sin barajar,
# para que los 5 modelos vean exactamente el mismo orden de chunks.
lr_results = {} # eta0 -> {'rmse': [...], 'diverged': bool, 'diverged_at': int

for eta0 in ETA0_VALUES:
    sgd_lr = SGDRegressor(loss='squared_error', learning_rate='constant', eta0=eta0)
    rmse = []
    diverged = False
    diverged_at = None

    for start in range(0, n_train_msd, CHUNK_SIZE):
        X_chunk = msd_scaler.transform(X_train_msd[start:start + CHUNK_SIZE])
        y_chunk = y_train_msd[start:start + CHUNK_SIZE]

        sgd_lr.partial_fit(X_chunk, y_chunk)

        chunk_rmse = np.sqrt(mean_squared_error(y_chunk, sgd_lr.predict(X_chunk)))
        rmse.append(chunk_rmse)

        # "Diverge" tanto si el RMSE deja de ser finito (NaN/inf) como si crece
        # a valores sin sentido físico (con eta0 grande, el float64 tarda varios
        # chunks en desbordar a inf, pero ya es evidente que el modelo explotó).
        if not np.isfinite(chunk_rmse) or chunk_rmse > DIVERGENCE_THRESHOLD:
            diverged = True
            diverged_at = len(rmse)
            break

    lr_results[eta0] = {'rmse': rmse, 'diverged': diverged, 'diverged_at': diverged_at

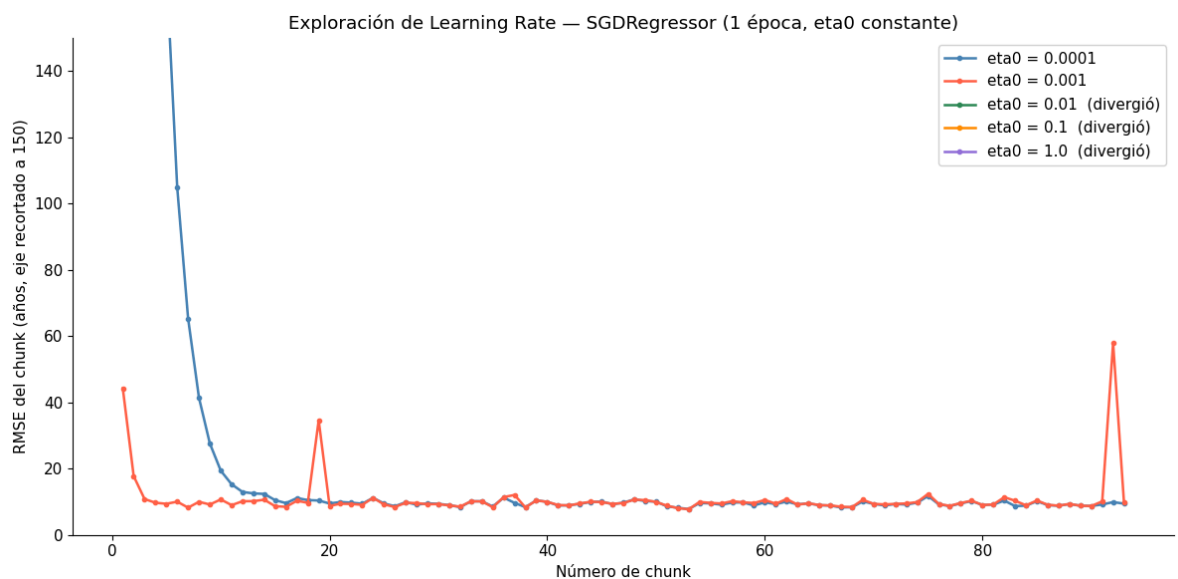
    if diverged:
        print(f'eta0 = {eta0:<7} → DIVERGIÓ en el chunk {diverged_at} / {n_chunks}
              f'(RMSE = {rmse[-1]:.2e} años)')
    else:
        print(f'eta0 = {eta0:<7} → completó la época sin diverger
              f'(RMSE final = {rmse[-1]:.4f} años)')
```

```
eta0 = 0.0001 → completó la época sin diverger (RMSE final = 9.4396 años)
eta0 = 0.001  → completó la época sin diverger (RMSE final = 9.7518 años)
eta0 = 0.01   → DIVERGIÓ en el chunk 1 / 93 (RMSE = 3.96e+09 años)
eta0 = 0.1    → DIVERGIÓ en el chunk 1 / 93 (RMSE = 5.01e+12 años)
eta0 = 1.0    → DIVERGIÓ en el chunk 1 / 93 (RMSE = 5.32e+13 años)
```

```
In [31]: fig, ax = plt.subplots(figsize=(12, 6))

for eta0, color in zip(ETA0_VALUES, COLORS):
    rmse = lr_results[eta0]['rmse']
    x_vals = np.arange(1, len(rmse) + 1)
    label = f'eta0 = {eta0}'
    if lr_results[eta0]['diverged']:
        label += ' (divergió)'
    ax.plot(x_vals, rmse, color=color, lw=1.8, marker='o', markersize=3, label=label)

ax.set_ylim(0, 150) # Las corridas divergentes se salen de este rango casi de inmediato
ax.set_xlabel('Número de chunk')
ax.set_ylabel('RMSE del chunk (años, eje recortado a 150)')
ax.set_title('Exploración de Learning Rate — SGDRegressor (1 época, eta0 constante)')
ax.legend()
plt.tight_layout()
plt.savefig('assets/sgd_learning_rate_exploration.png', dpi=120, bbox_inches='tight')
plt.show()
```



¿Qué learning rates divergen? ¿Por qué? ¿Qué cuesta un learning rate muy pequeño?

Cuáles divergen. De los cinco valores probados, $\eta_0 = 0.01$, 0.1 y 1.0 divergen y lo hacen literalmente en el primer chunk: el RMSE pasa de nada a miles de millones, billones e incluso decenas de billones de años en una sola actualización de pesos. Solo $\eta_0 = 0.0001$ y 0.001 completan la época con un RMSE final razonable (9.44 y 9.75 años).

Por qué un learning rate grande diverge. SGDRegressor actualiza los pesos en la dirección contraria al gradiente. Si pensamos en la superficie de la función de costo (un "tazón" con un mínimo), un paso de tamaño correcto nos deja un poco más cerca del fondo en cada actualización. Pero si η es demasiado grande, el paso es tan largo que salta por encima del mínimo y cae en un punto de la otra pared del tazón que está más alto que donde empezamos porque cerca del mínimo la superficie es curva, no plana, y un paso lineal basado en la pendiente de un solo punto sobrestima cuánto hay que moverse. En la siguiente iteración el gradiente es aún más grande (porque ahora estamos más lejos y más arriba), así que el siguiente paso es todavía más largo, y el proceso se retroalimenta: cada "rebote" es peor que el anterior. Esto es exactamente lo que se ve en

los datos no es un crecimiento gradual, es una explosión inmediata desde el primer chunk, porque el sobre-paso ya ocurre en la primera actualización.

Qué cuesta un learning rate muy pequeño. $\eta_0 = 0.0001$ no diverge, pero paga un costo claro en velocidad: necesita unos 15-18 chunks (casi el 20% de la época) para bajar de un RMSE inicial alto (fuera del rango de 150 años del gráfico) hasta estabilizarse cerca de 9-10 años, mientras que $\eta_0 = 0.001$ llega a esa misma zona en apenas 4-5 chunks. Si solo tuviéramos una fracción de los datos disponibles, un learning rate demasiado conservador podría terminar la única pasada de datos disponible sin haber convergido todavía la seguridad de "no vamos a divergir" se paga con una convergencia mucho más lenta.

En conjunto, esto confirma la intuición central de la Sección 4.3: η_0 es el hiperparámetro más sensible de SGD, y el rango que realmente funciona bien para este problema (features ya estandarizadas) es angosto entre 0.0001 y 0.001 aproximadamente, con 0.001 como el mejor compromiso entre velocidad y estabilidad de los cinco valores probados.

Exercise 2.5 — Learning Rate Schedules (Advanced)

A fixed learning rate is a compromise: large enough to converge quickly, small enough to avoid divergence. **Learning rate schedules** decay the rate over time, allowing fast initial progress and fine-grained convergence later.

sklearn's `SGDRegressor` supports several built-in schedules via `learning_rate`:

Schedule	Formula	Notes
'constant'	$\eta = \eta_0$	Fixed; need to tune η_0
'optimal'	$\eta_t = 1 / (\alpha (t + t_0))$	Auto-calculated; ignores η_0
'invscaling'	$\eta_t = \eta_0 / t^{\text{power_t}}$	Polynomial decay; flexible

Tasks:

1. Train three models for 3 epochs each: `learning_rate='constant'` ($\eta_0=0.001$), `learning_rate='optimal'`, and `learning_rate='invscaling'` ($\eta_0=0.001$, $\text{power_t}=0.25$).
2. After each epoch, record the validation RMSE on the test set.
3. Plot validation RMSE vs. epoch for all three schedules.
4. Answer: which schedule reaches the lowest validation RMSE? What is the trade-off between `constant` and decaying schedules in the context of streaming data (where the data distribution may shift over time)?

```
In [32]: N_EPOCHS_EX25 = 3

# Un diccionario de kwargs por schedule; 'optimal' ignora eta0, así que no se lo
schedule_configs = {
```

```

'constant': dict(learning_rate='constant', eta0=0.001),
'optimal': dict(learning_rate='optimal'),
'invscaling': dict(learning_rate='invscaling', eta0=0.001, power_t=0.25),
}

schedule_val_rmse = {} # nombre del schedule -> lista de RMSE de validación por

for name, kwargs in schedule_configs.items():
    sgd_sched = SGDRegressor(loss='squared_error', random_state=RANDOM_STATE, **
    # Misma semilla para los 3 schedules: así ven exactamente el mismo orden de
    # batches en cada época, y la comparación aísla el efecto del schedule solo.
    rng_sched = np.random.default_rng(RANDOM_STATE)
    val_rmse = []

    for epoch in range(N_EPOCHS_EX25):
        shuffled_idx = rng_sched.permutation(n_train_msd)

        for start in range(0, n_train_msd, CHUNK_SIZE):
            batch_idx = shuffled_idx[start:start + CHUNK_SIZE]
            X_chunk = msd_scaler.transform(X_train_msd[batch_idx])
            y_chunk = y_train_msd[batch_idx]
            sgd_sched.partial_fit(X_chunk, y_chunk)

        y_val_pred = sgd_sched.predict(X_test_msd_scaled)
        val_rmse = np.sqrt(mean_squared_error(y_test_msd, y_val_pred))
        val_rmse.append(val_rmse)

    schedule_val_rmse[name] = val_rmse
    rmse_str = ', '.join(f'{v:.4f}' for v in val_rmse)
    print(f'{name:<11} -> Val RMSE por época: {rmse_str}')

```

```

constant    -> Val RMSE por época: 9.9274, 10.0818, 10.0053
optimal     -> Val RMSE por época: 1031632047538.1833, 573848206953.2244, 21958444
618.4444
invscaling  -> Val RMSE por época: 9.5400, 9.5282, 9.5141

```

```

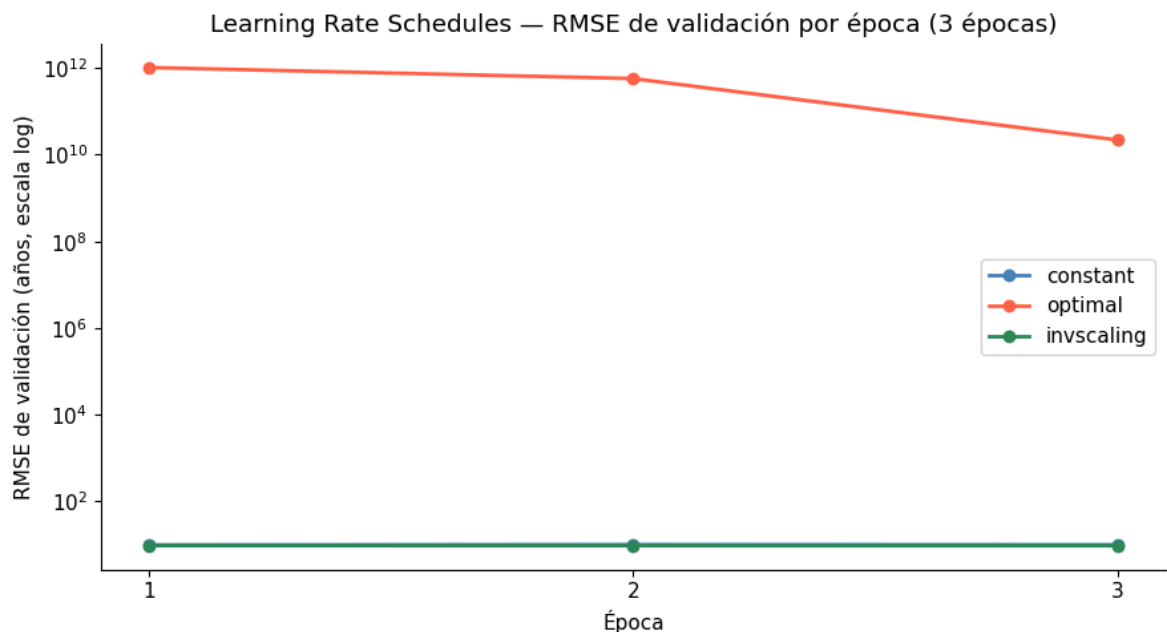
In [33]: fig, ax = plt.subplots(figsize=(9, 5))
epochs_x = np.arange(1, N_EPOCHS_EX25 + 1)

for (name, val_rmse), color in zip(schedule_val_rmse.items(), COLORS):
    ax.plot(epochs_x, val_rmse, color=color, marker='o', lw=2, label=name)

# Escala Logarítmica: 'optimal' diverge a valores ~1e12, muy por encima de los
# otros dos (~9-10). En escala lineal esa diferencia de magnitud aplastaría
# las otras dos curvas contra el eje X.
ax.set_yscale('log')
ax.set_xlabel('Época')
ax.set_ylabel('RMSE de validación (años, escala log)')
ax.set_xticks(epochs_x)
ax.set_title('Learning Rate Schedules - RMSE de validación por época (3 épocas)')
ax.legend()
plt.tight_layout()
plt.savefig('assets/sgd_lr_schedules.png', dpi=120, bbox_inches='tight')
plt.show()

print('Detalle (constant vs. invscaling, sin "optimal" que diverge):')
for name in ('constant', 'invscaling'):
    rmse_str = ', '.join(f'{v:.4f}' for v in schedule_val_rmse[name])
    print(f' {name:<11} -> {rmse_str}')

```



Detalle (constant vs. invscaling, sin "optimal" que diverge):

constant → 9.9274, 10.0818, 10.0053

invscaling → 9.5400, 9.5282, 9.5141

¿Qué schedule llega al menor RMSE de validación? ¿Cuál es el trade-off entre constant y schedules decrecientes en streaming con concept drift?

Cuál gana. De los tres, invscaling es el mejor con claridad: 9.54 → 9.53 → 9.51 años, mejorando levemente en cada época y terminando por debajo de constant (9.93 → 10.08 → 10.01, sin mejora consistente el mismo patrón de "rebote" que ya vimos en el Ejercicio 2.3). El caso más llamativo es optimal, que diverge por completo: arranca en $\sim 10^{12}$ años y va bajando ($10^{12} \rightarrow 5.7 \times 10^{11} \rightarrow 2.2 \times 10^{10}$) pero sigue siendo un resultado sin ningún sentido después de 3 épocas completas. Esto no es un error de código: `learning_rate='optimal'` calcula su propio tamaño de paso a partir del parámetro de regularización `alpha` (con un valor por defecto pensado para datos ya centrados en 0, como es típico en clasificación), y nuestro target real el año de la canción, con media ≈ 1998 está muy lejos de 0. El primer paso que calcula el schedule "optimal" es tan grande para esta escala de datos que el modelo explota desde la primera actualización, igual que vimos con `eta0` demasiado grande en el Ejercicio 2.4 solo que aquí el tamaño de paso "grande" lo eligió el propio algoritmo, no nosotros.

El trade-off constant vs. decrecientes en streaming con concept drift. Un schedule que decae (invscaling, o en general cualquier `n_t` que se hace mas pequeño con `t`) tiene una ventaja clara en un dataset fijo y estático como este: al principio da pasos grandes para acercarse rápido a una buena zona, y después los va acortando sin rebotar por eso termina con menor RMSE que constant en las mismas 3 épocas. Pero esa misma propiedad es un problema si los datos llegan como un stream real donde la distribución puede cambiar con el tiempo (concept drift, por ejemplo si de repente empiezan a llegar canciones de un género o época distintos a los que el modelo ya vio): después de muchas iteraciones, `n_t` se vuelve tan pequeño que el modelo ya casi no actualiza sus pesos, así que no puede reaccionar a ese cambio de distribución. constant en cambio, mantiene siempre la misma capacidad de adaptación nunca termina de asentarse con

precisión (como vimos en el Ejercicio 2.3), pero tampoco se queda "congelado" ante datos nuevos.

En definitiva: para un dataset fijo que se puede procesar varias veces sin que cambie, un schedule decreciente como invscaling es preferible porque converge a un error menor. Para un flujo de datos verdaderamente online con concept drift, constant (o un schedule que decaiga muy despacio) es la opción más segura, aunque nunca termine de converger del todo es el precio a pagar por mantenerse siempre listo para adaptarse. Y el caso de optimal es un recordatorio de que ningún schedule es automáticamente seguro, su comportamiento depende de supuestos sobre la escala de los datos que hay que validar antes de confiar en él.

5. Real-World Considerations

When to Use Batch vs. Online Learning

Scenario	Recommendation
Data fits in memory, static	Batch (Ridge/Lasso + Pipeline + GridSearchCV)
Data too large for RAM	Online (<code>SGDRegressor.partial_fit</code> on chunks)
Continuous data stream (concept drift)	Online with constant or slowly decaying LR
Periodic retraining on large archive	Mini-batch SGD or distributed frameworks (Spark MLlib, Dask-ML)

Practical Tips for SGD in Production

Always scale features. SGD is sensitive to feature scale; unscaled inputs can cause gradients to differ by orders of magnitude across features, making tuning nearly impossible. Fit the scaler once on a representative sample (or the first chunk) and apply it consistently.

Monitor for divergence early. Add a check after the first few chunks: if RMSE is not decreasing or is NaN, stop and reduce `eta0` by a factor of 10.

Regularization matters even more for SGD. With noisy mini-batch gradients, models can overfit if run for many epochs without regularization. Use `penalty='l2'` (default) with a non-zero `alpha` in `SGDRegressor`.

Warm-starting scaler for streaming data. If your data distribution shifts over time, consider re-fitting the scaler periodically (e.g., every N chunks) or using `StandardScaler` with `partial_fit` to update statistics incrementally.

Evaluation on held-out data. The per-chunk train RMSE is noisy (gradient descent is still mid-step). Always evaluate on a separately held-out test set for a reliable performance estimate.

Pipelines Are Not Directly Compatible with `partial_fit`

sklearn's `Pipeline` object does not support `partial_fit` in all configurations — specifically, you cannot call `pipeline.partial_fit()` if the pipeline contains transformers that need to be fitted incrementally. The typical workaround (as implemented above) is to manage the scaler manually outside the loop and feed pre-scaled chunks to the SGD estimator.

6. Summary and Key Takeaways

Part 1 — Pipelines and Regularization

- **Pipelines** chain preprocessing and modeling steps into a single object that integrates cleanly with `cross_val_score` and `GridSearchCV`, preventing data leakage.
- **Polynomial features** extend a linear model's capacity to capture non-linear relationships. The expansion is controlled by `degree`; higher degrees increase variance and require stronger regularization.
- **Ridge regression** adds an L2 penalty to the OLS loss: $\min \|Xw - y\|^2 + \alpha \|w\|^2$. It is always uniquely solvable and numerically stable, even with many correlated features.
- **GridSearchCV** evaluates each candidate α using k-fold CV and returns the optimal value. The validation curve shows the classic U-shape: decreasing CV error as α grows (fighting overfitting), then increasing error as α becomes too large (causing underfitting).
- **Lasso** (L1) promotes sparsity — many coefficients become exactly zero — making it useful for feature selection when interpretability matters.

Part 2 — Online Learning with SGD

- **Mini-batch SGD** updates model weights on small chunks of data without ever loading the full dataset. It is the standard approach for datasets that exceed available RAM or for continuously arriving data.
- `SGDRegressor.partial_fit` incrementally updates weights without resetting them. Call it repeatedly on successive chunks or multiple epochs to approximate full-batch training.
- **The learning rate is critical.** Too large → gradient updates overshoot the loss minimum → divergence. Too small → convergence is impractically slow. The `'optimal'` schedule or a constant rate around `eta0 = 0.01` is a good starting point for standardized features.
- **Decaying schedules** (`invscaling`) are good for static datasets but can become too conservative for streaming data with concept drift; prefer `constant` or `optimal` in those settings.

